



UNIVERSITAS INDONESIA

**EFISIENSI EKSPLORASI PADA PERMAINAN MINESWEEPER
MENGUNAKAN METODE CRITIC AS EXPLORER (CAE)
DALAM KERANGKA DEEP Q-NETWORK (DQN)**

SKRIPSI

**YOEL DWI MIRYANO
2206059534**

**FAKULTAS TEKNIK
PROGRAM STUDI S1 TEKNIK KOMPUTER
DEPOK
2025**



UNIVERSITAS INDONESIA

**EFISIENSI EKSPLORASI PADA PERMAINAN MINESWEEPER
MENGUNAKAN METODE CRITIC AS EXPLORER (CAE)
DALAM KERANGKA DEEP Q-NETWORK (DQN)**

SKRIPSI

**Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Teknik**

**YOEL DWI MIRYANO
2206059534**

**FAKULTAS TEKNIK
PROGRAM STUDI S1 TEKNIK KOMPUTER
DEPOK
JULI 2025**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya saya sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah saya nyatakan dengan benar.**

Nama : Yoel Dwi Miryano

NPM : 2206059534

Tanda Tangan :

Tanggal : XX April 2026

HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL

Saya menyatakan bahwa dalam penyusunan Tugas Akhir ini, saya menggunakan teknologi Kecerdasan Buatan dengan platform Deepseek untuk tujuan pemeriksaan dan perbaikan tata bahasa dan saya bertanggung jawab penuh atas keakuratan, orisinalitas, dan integritas seluruh isi Tugas Akhir ini.

Nama : Yoel Dwi Miryano

NPM : 2206059534

Tanda Tangan :

Tanggal : XX April 2026

HALAMAN PENGESAHAN

Skripsi ini diajukan oleh :
Nama : Yoel Dwi Miryano
NPM : 2206059534
Program Studi : S1 Teknik Komputer
Judul Skripsi : Efisiensi Eksplorasi pada Permainan Minesweeper
menggunakan Metode Critic as Explorer (CAE)
dalam Kerangka Deep Q-Network (DQN)

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Teknik pada Program Studi S1 Teknik Komputer, Fakultas Teknik, Universitas Indonesia.

DEWAN PENGUJI

Pembimbing : Muhammad Firdaus Syawaludin Lubis, S.T., M.T., Ph.D. ()
Penguji : Yan Maraden, S.T., M.T., M.Sc. ()
Penguji : Alfian Prasekal, S.T., M.Sc., Ph.D. ()

Ditetapkan di : Depok
Tanggal : xx Maret 2026

KATA PENGANTAR

Puji dan syukur Penulis panjatkan kepada Tuhan Yang Maha Esa atas berkat dan rahmat-Nya lah Penulis dapat menyelesaikan Skripsi dengan judul “Efisiensi Eksplorasi pada Permainan Minesweeper menggunakan Metode Critic as Explorer (CAE) dalam Kerangka Deep Q-Network (DQN)”. Skripsi ini Penulis laksanakan sebagai salah satu syarat dalam memperoleh gelar Sarjana Teknik pada program studi S1 Teknik Komputer, Fakultas Teknik, Universitas Indonesia.

Penulis juga ingin menyampaikan rasa terima kasihnya kepada:

1. Muhammad Firdaus Syawaludin Lubis, S.T., M.T., Ph.D., selaku Dosen Pembimbing Skripsi Penulis.
2. Dr. Ir. Muhammad Salman, S.T., M.I.T., selaku Ketua Departemen Teknik Elektro, Fakultas Teknik, Universitas Indonesia.
3. Dr. Prima Dewi Purnamasari, S.T., M.Sc., selaku Kepala Program Studi S1 Teknik Komputer, Fakultas Teknik, Universitas Indonesia.
4. Prof. Dr.-Ing. Ir. Kalamullah Ramli, M.Eng., selaku Pembimbing Akademis Penulis.
5. Yan Maraden, S.T., M.T., M.Sc. dan Alfian Prasekal, S.T., M.Sc., Ph.D. selaku Dosen Penguji Skripsi Penulis.
6. Segenap dosen Departemen Teknik Elektro atas semua ilmu yang diberikan kepada Penulis.
7. Orang tua yang selalu memberikan dukungan serta doa, sehingga penulisan Skripsi dapat diselesaikan secara baik.

8. Teman-teman Teknik Komputer, selaku rekan penulis yang selalu membantu serta memberikan dukungan moril sepanjang proses kuliah serta penulisan proposal skripsi.

Penulis menyadari dan menerima bahwa Skripsi yang dibentuk ini masih jauh dari sempurna dan masih memiliki berbagai kekurangan serta kesalahan baik dari segi ilmu maupun pemahaman. Oleh karena itu, Penulis mengharapkan segala kritik dan saran yang diberikan agar ke depannya Penulis dapat menjadi lebih baik.

XX April 2026

Yoel Dwi Miryano

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, saya yang bertanda tangan di bawah ini:

Nama : Yoel Dwi Miryano
NPM : 2206059534
Program Studi : S1 Teknik Komputer
Fakultas : Teknik
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah saya yang berjudul:

Efisiensi Eksplorasi pada Permainan Minesweeper menggunakan Metode Critic as Explorer (CAE) dalam Kerangka Deep Q-Network (DQN)

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir saya selama tetap mencantumkan nama saya sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini saya buat dengan sebenarnya.

Dibuat di : Depok
Pada tanggal : XX April 2026
Yang menyatakan

(Yoel Dwi Miryano)
vii

ABSTRAK

Nama : Yoel Dwi Miryano
Program Studi : S1 Teknik Komputer
Judul : Efisiensi Eksplorasi pada Permainan Minesweeper menggunakan Metode Critic as Explorer (CAE) dalam Kerangka Deep Q-Network (DQN)
Pembimbing : Muhammad Firdaus Syawaludin Lubis, S.T., M.T., Ph.D.

Reinforcement Learning dengan arsitektur *Deep Q-Network* telah terbukti efektif dalam berbagai *environment* pengambilan keputusan. Namun, strategi eksplorasi standar ϵ -greedy menunjukkan keterbatasan pada *environment* dengan observasi parsial (POMDP), *reward* yang jarang, dan penalti terminal yang fatal, seperti permainan *Minesweeper*, karena eksplorasi acak cenderung menyebabkan terminasi dini dan membatasi akumulasi pengalaman yang informatif. Penelitian ini menginvestigasi integrasi metode *Critic as Explorer* (CAE), yaitu strategi eksplorasi berbasis ketidakpastian yang memanfaatkan representasi laten dari jaringan *critic*, ke dalam kerangka *Deep Q-Network* untuk meningkatkan efisiensi sampel. Eksperimen dilakukan pada papan *Minesweeper* berukuran 6×6 dengan 4 ranjau, menggunakan dua *agent* dengan arsitektur *Convolutional Neural Network* (CNN) yang identik. *Deep Q-Network* menggunakan *policy* ϵ -greedy sebagai *baseline*, sedangkan *Critic as Explorer-Deep Q-Network* dengan bonus eksplorasi berbasis *Upper Confidence Bound* bertindak sebagai metode yang diusulkan. Kedua *agent* dilatih selama 80,000 episode dan dievaluasi pada lima *seed* papan berbeda untuk mengukur kemampuan generalisasi. Hasil penelitian menunjukkan bahwa *Critic as Explorer-Deep Q-Network* secara signifikan mengungguli *baseline*, dengan rata-rata *win rate* sebesar 69.6% dibandingkan 58.0% ($p < 0.001$, $d = 6.00$), serta efisiensi sampel yang lebih tinggi, ditunjukkan dengan pencapaian *win rate* 50% dalam 6,000 episode dibandingkan 65,000 episode. Analisis lebih lanjut mengindikasikan bahwa bonus eksplorasi mengalami *decay* secara adaptif (*self-annealing*) tanpa penjadwalan manual dan menghasilkan eksplorasi yang lebih terarah, dibuktikan melalui peningkatan metrik *progress per click* sebesar 12.1%. Meskipun terjadi degradasi performa pada fase pelatihan lanjut akibat *policy collapse* yang disebabkan oleh terhapusnya data eksplorasi awal yang beragam dari *replay buffer*, model yang disimpan pada titik performa puncak tetap menunjukkan kemampuan generalisasi yang kuat. Temuan ini menunjukkan bahwa eksplorasi berbasis ketidakpastian merupakan strategi yang efektif untuk *environment* dengan observasi parsial, *reward* jarang, dan penalti fatal, serta menyediakan landasan empiris bagi pengembangan *agent* yang lebih adaptif pada domain berbasis logika.

Kata kunci:

Reinforcement Learning, *Deep Q-Network*, Eksplorasi, *Critic as Explorer*, *Minesweeper*, *Upper Confidence Bound*, Efisiensi Sampel.

ABSTRACT

Name : Yoel Dwi Miryano
Study Program : S1 Teknik Komputer
Title : Exploration Efficiency in Minesweeper using Critic as Explorer (CAE) within a Deep Q-Network Framework
Supervisor : Muhammad Firdaus Syawaludin Lubis, S.T., M.T., Ph.D.

Reinforcement Learning with *Deep Q-Network* architectures has been proven effective in various decision-making environments. However, the standard ϵ -greedy exploration strategy exhibits limitations in *partially observable environment* (POMDP) with sparse rewards and fatal terminal penalties, such as *Minesweeper*, since random exploration often leads to early termination and restricts the accumulation of informative experiences. This study investigates the integration of the *Critic as Explorer* (CAE) method, an uncertainty-based exploration strategy that leverages the latent representations of the critic network, into the *Deep Q-Network* framework to improve sample efficiency. Experiments were conducted on a 6×6 Minesweeper board with 4 mines, using two agents with identical *Convolutional Neural Network* (CNN) architectures. Deep Q-Network uses an ϵ -greedy policy as the baseline, while *Critic as Explorer-Deep Q-Network*, augmented with an Upper Confidence Bound exploration bonus, acts as the proposed method. Both agents were trained for 80,000 episodes and evaluated on five different board seeds to measure generalization. Results show that *Critic as Explorer-Deep Q-Network* significantly outperforms the baseline, with an average win rate of 69.6% compared to 58.0% ($p < 0.001$, $d = 6.00$), as well as higher sample efficiency, achieving a 50% win rate in 6,000 episodes compared to 65,000 episodes. Further analysis indicates that the exploration bonus adaptively decays (self-annealing) without manual scheduling and yields more directed exploration, shown by a 12.1% increase in the progress per click metric. Although performance degradation occurred in the later stages of training due to policy collapse caused by the eviction/erasure of diverse early-exploration data from the replay buffer, the model saved at peak performance maintained strong generalization. These findings demonstrate that uncertainty-based exploration is an effective strategy for environments with partial observability, sparse rewards, and fatal penalties, and provides an empirical foundation for the development of more adaptive agents in logic-based domains.

Keywords: Reinforcement Learning, Deep Q-Network, Exploration, Critic as Explorer, Minesweeper, Upper Confidence Bound, Sample Efficiency.

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN PERNYATAAN ORISINALITAS	ii
HALAMAN PERNYATAAN PENGGUNAAN KECERDASAN ARTIFISIAL	iii
HALAMAN PENGESAHAN	iv
KATA PENGANTAR	v
HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI	vii
ABSTRAK	viii
ABSTRACT	ix
DAFTAR ISI	x
DAFTAR TABEL	xiii
DAFTAR GAMBAR	xiv
DAFTAR PERSAMAAN	xv
DAFTAR NOTASI	xvi
DAFTAR KODE PROGRAM	xvii
DAFTAR SINGKATAN	xviii
1 PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	3
1.4 Batasan Masalah	4
1.5 Manfaat Penelitian	4
1.5.1 Manfaat Praktis	5
1.5.2 Manfaat Akademis	5
1.6 Metodologi Penelitian	5
1.7 Sistematika Penulisan	7
2 TINJAUAN PUSTAKA	8
2.1 Dasar Reinforcement Learning	8
2.1.1 Arsitektur Deep Q-Network (DQN)	9
2.1.2 DQN sebagai Baseline Penelitian	10

2.2	Masalah Eksplorasi pada DQN	11
2.3	Minesweeper: Tantangan Logika dan Observasi Parsial	12
2.4	Studi Terdahulu dan Posisi Penelitian	13
2.4.1	RL untuk Permainan Minesweeper	14
2.4.2	Strategi Eksplorasi Berbasis Ketidakpastian	14
2.4.3	Kesenjangan dan Motivasi Penelitian	15
2.5	Strategi Eksplorasi Critic as Explorer (CAE)	16
2.5.1	Prinsip Kerja CAE	17
2.5.2	Mekanisme Bonus Ketidakpastian	18
2.6	Kontribusi Penelitian	19
3	PERANCANGAN DAN IMPLEMENTASI SISTEM	21
3.1	Prosedur Pelatihan	21
3.2	Desain Environment	22
3.2.1	Konfigurasi Papan dan Ranjau	22
3.2.2	Representasi State: One-Hot Encoding	22
3.2.3	Struktur Reward	24
3.2.4	Kondisi Terminasi dan Observabilitas Parsial	24
3.2.5	Mekanisme Tambahan	25
3.3	Desain Algoritma	25
3.3.1	DQN	26
3.3.2	CAE-DQN	30
3.4	Pelatihan	33
3.5	Evaluasi	33
3.6	Metode Analisis	34
4	EKSPERIMEN DAN ANALISIS	36
4.1	Data dan Hasil	36
4.1.1	Kurva Pelatihan	36
4.1.2	Kurva Evaluasi Pelatihan	39
4.1.3	Bonus Eksplorasi CAE	39
4.1.4	Metrik Evaluasi	40
4.2	Analisis	41
4.2.1	Perbandingan Performa Akhir	42
4.2.2	Efisiensi Sampel dan Kecepatan Pembelajaran	43
4.2.3	Adaptivitas Eksplorasi: Self-Annealing Bonus CAE	45
4.2.4	Kualitas Eksplorasi: Progress per Click	45
4.2.5	Stabilitas Pelatihan dan Fenomena Penurunan	46
4.2.6	Ringkasan Temuan Kuantitatif	48
4.3	Pembahasan	50
5	PENUTUP	52
5.1	Saran	53
	DAFTAR REFERENSI	55

LAMPIRAN	1
---------------------------	----------

DAFTAR TABEL

Tabel 3.1	Struktur <i>Reward Environment Minesweeper</i>	24
Tabel 3.2	Ringkasan Konfigurasi <i>Environment</i>	25
Tabel 3.3	Ringkasan <i>Hyperparameter</i> dan Konfigurasi <i>Deep Q-Network</i> (DQN)	29
Tabel 3.4	Ringkasan <i>Hyperparameter</i> dan Konfigurasi <i>Critic as Explorer</i> (CAE)-DQN	32
Tabel 3.5	Konfigurasi Pelatihan dan Pencatatan Data	33
Tabel 3.6	Konfigurasi Evaluasi Akhir	34
Tabel 3.7	Metode Analisis Kuantitatif yang Digunakan	35
Tabel 4.1	Rangkuman metrik evaluasi model DQN dan CAE-DQN sebanyak 500 episode per <i>seed</i>	41
Tabel 4.2	Ringkasan Temuan Kuantitatif: Perbandingan DQN dan CAE-DQN	49

DAFTAR GAMBAR

Gambar 2.1	Arsitektur konseptual CAE [1]	17
Gambar 3.1	Diagram alur prosedur pelatihan secara umum	21
Gambar 3.2	Contoh representasi <i>One-Hot Encoding</i> dari <i>Minesweeper</i> 6×6	23
Gambar 3.3	Arsitektur <i>Convolutional Neural Network</i> (CNN) dari DQN pada Eksperimen	26
Gambar 3.4	Diagram alur pelatihan DQN	28
Gambar 3.5	Arsitektur CNN dari CAE-DQN pada Eksperimen	30
Gambar 3.6	Diagram alur pelatihan CAE-DQN. Proses setelah <code>agent.train(done)</code> sama dengan DQN namun tanpa mekanisme ϵ -decay.	31
Gambar 4.1	<i>Total reward</i> dari pelatihan DQN dan CAE-DQN selama 80,000 episode	37
Gambar 4.2	<i>Number of clicks</i> dari pelatihan DQN dan CAE-DQN selama 80,000 episode	38
Gambar 4.3	Kurva <i>win rate - time</i> dari pelatihan DQN dan CAE-DQN selama 80,000 episode	39
Gambar 4.4	<i>Self-annealing</i> bonus eksplorasi CAE selama 20,000 episode pelatihan.	40

DAFTAR PERSAMAAN

(2.1)	Expected Discounted Return	8
(2.2)	Target DQN	10
(2.3)	Loss Function DQN	10
(2.4)	ϵ -greedy DQN	11
(2.5)	Bonus Eksplorasi CAE	18
(2.6)	Q-Value Teraugmentasi CAE	18

DAFTAR NOTASI

s_t	State (keadaan) pada waktu t
a_t	Action (aksi) yang dipilih pada waktu t
r_t	Reward (imbalan) skalar yang diterima
π	Policy (kebijakan) agent
γ	Faktor diskon (<i>discount factor</i>), $\gamma \in [0, 1)$
G_t	<i>Expected discounted return</i> mulai dari waktu t
$Q(s, a)$	Fungsi nilai aksi (<i>action-value function</i>)
θ	Parameter jaringan saraf utama (<i>online network</i>)
θ^-	Parameter jaringan target (<i>target network</i>)
y_t	Nilai target untuk pembaruan (<i>TD-target</i>)
$\mathcal{D}$	<i>Replay buffer</i>
ϵ	Probabilitas eksplorasi pada strategi ϵ -greedy
$\phi(s)$	Vektor fitur laten (<i>embedding</i>) dari jaringan <i>critic</i>
A	<i>Gram matrix</i> yang menyimpan informasi kovarians fitur laten
A^{-1}	Invers dari <i>Gram matrix</i>
β	Koefisien eksplorasi (<i>exploration bonus coefficient</i>)
λ	Parameter regularisasi <i>ridge</i> untuk inisialisasi matriks A
$b(s)$	Bonus eksplorasi intrinsik berbasis ketidakpastian
$Q^{\text{aug}}(s, a)$	Q -value yang telah diaugmentasi dengan bonus $b(s)$
p	p -value (signifikansi statistik)
d	<i>Cohen's d</i> (ukuran efek / <i>effect size</i>)
W	Statistik uji <i>Shapiro-Wilk</i> (uji normalitas)
t	Statistik uji <i>Welch's t-test</i>
R^2	Koefisien determinasi (regresi linier)
SD	Standar deviasi
α	<i>Learning rate</i> (pada optimizer ADAM)
Σ	Notasi untuk penjumlahan (sigma)

DAFTAR KODE PROGRAM

DAFTAR SINGKATAN

RL	<i>Reinforcement Learning</i>
POMDP	<i>Partially Observable Markov Decision Process</i>
MDP	<i>Markov Decision Process</i>
TD	<i>Temporal Difference</i>
MSE	<i>Mean Squared Error</i>
DQN	<i>Deep Q-Network</i>
PPO	<i>Proximal Policy Optimization</i>
SGD	<i>Stochastic Gradient Descent</i>
CSP	<i>Constraint Satisfaction Problem</i>
CNN	<i>Convolutional Neural Network</i>
ReLU	<i>Rectified Linear Unit</i>
LR	<i>Learning Rate</i>
ADAM	<i>Adaptive Moment Estimation</i>
CAE	<i>Critic as Explorer</i>
UCB	<i>Upper Confidence Bound</i>
SMA	<i>Simple Moving Average</i>
AUC	<i>Area Under Curve</i>
NN	<i>Neural network</i>
ML	<i>Machine Learning</i>
IID	Independensi dan Distribusi Identik
SAC	<i>Soft Actor Critic</i>
TD3	<i>Twin Delayed Deep Deterministic Policy Gradient</i>
MAB	<i>Multi-Armed Bandit</i>
FIFO	<i>First-In-First-Out</i>

BAB 1

PENDAHULUAN

Bab ini menyajikan landasan penelitian yang mencakup latar belakang, perumusan masalah, tujuan, ruang lingkup, manfaat, metodologi, serta sistematika penulisan. Bagian ini memberikan gambaran umum mengenai arah dan kontribusi utama penelitian.

1.1 Latar Belakang

Reinforcement Learning (RL) menyediakan kerangka kerja bagi *agent* untuk mempelajari *policy* melalui interaksi langsung dengan *environment*. DQN [2] merupakan salah satu pendekatan yang paling luas digunakan karena kemampuannya dalam mengaproksimasi *value function* dari representasi *state* mentah. Namun, performa DQN sangat bergantung pada strategi eksplorasi yang digunakan. Pendekatan standar ϵ -greedy, yang mengandalkan pemilihan *action* acak dengan probabilitas tetap atau terjadwal, bersifat tidak efisien pada *environment* dengan *reward* jarang dan penalti terminal. Dalam skenario tersebut, eksplorasi acak cenderung menghasilkan terminasi dini, membatasi kualitas dan keragaman pengalaman yang diperoleh selama pelatihan.

Permasalahan ini menjadi semakin nyata pada permainan *Minesweeper*, yang secara inheren menggabungkan *reward sparsity*, penalti fatal, dan observasi parsial. Setiap *action* yang keliru berujung pada terminasi episode, sementara keputusan yang benar memerlukan inferensi logis dari informasi lokal yang terbatas. Karakteristik ini secara alami memposisikan *Minesweeper* sebagai *Partially Observable Markov Decision Process* (POMDP) [3]. Dalam konteks tersebut, eksplorasi berbasis ϵ -greedy tidak efisien, dan secara sistematis gagal memaparkan *agent* terhadap konfigurasi *state* yang informatif [4].

Berbagai pendekatan eksplorasi berbasis motivasi intrinsik telah diusulkan untuk mengatasi keterbatasan ini, namun umumnya mengorbankan kesederhanaan melalui penambahan modul, parameter, atau objektif pelatihan tambahan. CAE [1] mengusulkan mekanisme eksplorasi yang ringan dengan memanfaatkan representasi laten dari jaringan

critic untuk mengestimasi ketidakpastian berbasis *Upper Confidence Bound* (UCB). Mekanisme ini mendorong eksplorasi pada *state* yang kurang terwakili tanpa memperkecilkan parameter pelatihan tambahan, sehingga tetap mempertahankan efisiensi komputasi. Meskipun menunjukkan hasil yang kompetitif pada domain kontrol kontinu dan navigasi spasial, generalisasi pendekatan ini pada *environment* berbasis logika dengan penalti terminal, seperti *Minesweeper*, masih belum teruji.

Berdasarkan kesenjangan tersebut, penelitian ini menginvestigasi efektivitas integrasi CAE ke dalam kerangka DQN pada permainan *Minesweeper*. Secara khusus, penelitian ini mengevaluasi apakah eksplorasi berbasis ketidakpastian laten mampu meningkatkan efisiensi sampel, menghasilkan dinamika eksplorasi yang adaptif (*self-annealing*), serta mendorong eksplorasi yang lebih terarah dibandingkan dengan ϵ -greedy. Dengan demikian, penelitian ini tidak hanya menguji kinerja metode yang diusulkan, tetapi juga memberikan pemahaman empiris mengenai peran eksplorasi berbasis representasi laten dalam *environment* dengan struktur logika yang kompleks dan risiko penalti tinggi.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, penelitian ini difokuskan pada evaluasi efektivitas eksplorasi berbasis ketidakpastian laten dalam kerangka DQN. Secara khusus, permasalahan penelitian dirumuskan sebagai berikut:

1. Validitas Implementasi (*Correctness*)

Apakah implementasi CAE-DQN mampu merealisasikan mekanisme eksplorasi berbasis ketidakpastian sebagaimana diusulkan dalam [1] secara konsisten dan sesuai dengan formulasi aslinya?

2. Efisiensi Sampel (*Performance*)

Apakah CAE-DQN mampu mencapai kinerja yang lebih tinggi, diukur melalui *win rate*, dengan jumlah episode pelatihan yang lebih sedikit dibandingkan dengan DQN berbasis ϵ -greedy pada permainan *Minesweeper*?

3. Adaptivitas Eksplorasi (*Mechanism*)

Apakah bonus eksplorasi pada **CAE** secara alami mengalami penurunan (*self-annealing*) seiring bertambahnya pengalaman, menghilangkan kebutuhan akan skema *exploration decay* yang ditentukan secara manual?

4. **Kualitas Eksplorasi (*Behavioral Quality*)**

Apakah eksplorasi yang dipandu oleh **CAE** menghasilkan *action* yang lebih informatif, yang tercermin dari progres permainan per langkah yang lebih tinggi, dibandingkan dengan eksplorasi acak pada ϵ -greedy, khususnya pada tahap awal pelatihan?

1.3 Tujuan Penelitian

Penelitian ini bertujuan untuk mengevaluasi secara sistematis efektivitas eksplorasi berbasis ketidakpastian laten dalam kerangka DQN pada permainan *Minesweeper*. Secara khusus, tujuan penelitian ini adalah sebagai berikut:

1. **Memvalidasi Implementasi (*Correctness*)**

Merealisasikan dan memverifikasi implementasi **CAE-DQN** agar sesuai dengan mekanisme eksplorasi berbasis ketidakpastian yang diusulkan [1].

2. **Mengevaluasi Efisiensi Sampel (*Performance*)**

Menganalisis kemampuan **CAE-DQN** dalam mencapai kinerja yang lebih tinggi, diukur melalui *win rate*, dengan jumlah episode pelatihan yang lebih sedikit dibandingkan dengan DQN berbasis ϵ -greedy.

3. **Menganalisis Adaptivitas Eksplorasi (*Mechanism*)**

Menginvestigasi apakah bonus eksplorasi pada **CAE** menunjukkan perilaku *self-annealing* secara alami seiring bertambahnya pengalaman, tanpa memerlukan skema *exploration decay* yang ditentukan secara manual.

4. **Menilai Kualitas Eksplorasi (*Behavioral Quality*)**

Mengevaluasi apakah eksplorasi yang dipandu oleh **CAE** menghasilkan *action* yang lebih informatif, yang tercermin dari progres permainan per langkah yang

lebih tinggi dibandingkan dengan eksplorasi acak pada ϵ -greedy, khususnya pada tahap awal pelatihan.

1.4 Batasan Masalah

Untuk menjaga fokus dan keterkendalian eksperimen, penelitian ini dibatasi pada ruang lingkup berikut:

1. *Environment* yang digunakan adalah permainan *Minesweeper* dengan ukuran papan tetap 6×6 dan jumlah ranjau sebanyak 4.
2. Representasi *state* dibatasi pada skema *one-hot encoding* dengan dimensi $6 \times 6 \times 10$ kanal.
3. Perbandingan metode difokuskan pada DQN berbasis ϵ -greedy dan **CAE-DQN** dengan bonus ketidakpastian berbasis UCB.
4. Arsitektur jaringan pada kedua *agent* dibuat identik, terdiri atas tiga lapisan konvolusional dan dua lapisan *fully connected*, dengan jumlah parameter yang setara.
5. Proses pelatihan dilakukan selama 80,000 episode. Untuk menjamin reproduisibilitas, *seed generator* angka acak diinisialisasi ke 2004, namun tata letak ranjau (*mine placement*) diacak ulang pada setiap awal episode. Model diinisialisasi dengan seed 123.
6. Parameter CAE ditetapkan konstan ($\beta = 0.1$ dan $\lambda = 1.0$) tanpa penerapan skema penjadwalan selama pelatihan.
7. Metrik evaluasi dibatasi pada *win rate*, *median progress*, *median reward* per episode, serta jumlah klik per episode.

1.5 Manfaat Penelitian

Penelitian ini diharapkan memberikan kontribusi yang signifikan, baik secara praktis maupun akademis, dalam pengembangan strategi eksplorasi pada RL, khususnya pada *environment* dengan penalti terminal dan *reward* yang jarang.

1.5.1 Manfaat Praktis

- Menyediakan implementasi **CAE-DQN** yang terverifikasi sebagai referensi bagi penelitian lanjutan maupun penerapan pada domain dengan karakteristik serupa.
- Menyajikan bukti empiris mengenai efektivitas eksplorasi berbasis ketidakpastian dalam meningkatkan kualitas eksplorasi dibandingkan pendekatan acak pada *environment* berisiko tinggi dengan *reward* jarang.
- Menunjukkan potensi CAE dalam mengurangi ketergantungan terhadap penjadwalan *hyperparameter* eksplorasi secara manual, menyederhanakan proses *tuning*.

1.5.2 Manfaat Akademis

- Mengisi kesenjangan dalam literatur melalui evaluasi CAE pada *environment logic-dense* yang dimodelkan sebagai POMDP dengan penalti terminal.
- Memberikan analisis kualitatif terhadap perilaku bonus CAE dalam konteks pengambilan keputusan berbasis inferensi logis.
- Mendokumentasikan dan mengevaluasi teknik implementasi praktis, seperti pembaruan *Sherman–Morrison* [5] dan normalisasi *Welford* [6], dalam konteks penerapan RL.

1.6 Metodologi Penelitian

Penelitian ini menggunakan pendekatan kuantitatif dengan desain eksperimental komparatif untuk mengevaluasi efektivitas eksplorasi berbasis ketidakpastian laten dalam kerangka DQN. Metodologi penelitian terdiri dari tahapan berikut:

1. Studi Literatur

Kajian dilakukan terhadap konsep RL, DQN, POMDP, permainan *Minesweeper*, serta metode eksplorasi CAE untuk membangun landasan teoretis dan mengidentifikasi posisi penelitian terhadap *state-of-the-art*.

2. Perancangan *Environment* dan *Agent*

Environment Minesweeper diimplementasikan dalam format *Gym-like* dengan konfigurasi papan 6×6 dan 4 ranjau. Dua *agent* dirancang, yaitu DQN berbasis ϵ -greedy sebagai *baseline* dan **CAE-DQN** dengan mekanisme bonus ketidakpastian berbasis UCB, dengan arsitektur jaringan yang identik untuk memastikan perbandingan yang adil.

3. Implementasi

Sistem diimplementasikan menggunakan *Python* dengan library *TensorFlow* dan *NumPy*. Komponen CAE, termasuk pembaruan *Gram Matrix* menggunakan *Sherman–Morrison* dan normalisasi *Welford*, diimplementasikan untuk menjaga efisiensi komputasi dan stabilitas numerik.

4. Pelatihan dan Evaluasi

Kedua *agent* dilatih selama 80,000 episode dengan *seed environment* tetap (2004) dan *seed model* tetap. Selama pelatihan, evaluasi dilakukan setiap 1,000 episode dengan mencatat metrik kinerja berdasarkan 1,000 episode terakhir. Setelah pelatihan, model terbaik dievaluasi pada lima *seed* papan berbeda (1,2,3,4,5) untuk mengukur kemampuan generalisasi. Metrik evaluasi meliputi *win rate*, *median progress*, dan jumlah klik per episode.

5. Analisis Hasil

Analisis hasil diarahkan secara sistematis untuk menjawab keempat rumusan masalah penelitian. Validitas implementasi CAE dievaluasi melalui konsistensi pembaruan *Gram Matrix* serta kestabilan normalisasi bonus sepanjang pelatihan, guna memastikan kesesuaian dengan formulasi asli. Efisiensi sampel dianalisis dengan membandingkan *win rate* terhadap jumlah episode pelatihan, untuk menguji apakah **CAE-DQN** mampu mencapai kinerja yang lebih tinggi dengan jumlah sampel yang lebih sedikit dibandingkan *baseline*. Adaptivitas eksplorasi dievaluasi secara kuantitatif melalui pengamatan terhadap penurunan alami (*self-annealing*) bonus CAE seiring bertambahnya pengalaman, sebagai indikator eliminasi kebutuhan akan skema *exploration decay* manual. Selanjutnya, kualitas eksplorasi dinilai berdasarkan progres permainan per langkah pada fase awal pelatihan, untuk

memverifikasi bahwa *action* yang dipilih oleh CAE lebih informatif dibandingkan eksplorasi acak berbasis ϵ -greedy.

1.7 Sistematika Penulisan

Laporan penelitian ini disusun dengan sistematika sebagai berikut:

1. Bab I – Pendahuluan

Bab ini memaparkan latar belakang, perumusan masalah, tujuan, batasan, manfaat, metodologi, serta sistematika penulisan. Bab ini memberikan gambaran umum mengenai arah dan kontribusi penelitian.

2. Bab II – Tinjauan Pustaka

Bab ini menyajikan landasan teoretis yang mendasari penelitian, meliputi konsep dasar RL, arsitektur DQN, permasalahan eksplorasi pada DQN, karakteristik *Minesweeper* sebagai POMDP, studi terdahulu, serta mekanisme eksplorasi CAE.

3. Bab III – Perancangan dan Implementasi Sistem

Bab ini menguraikan perancangan dan implementasi sistem, mencakup desain *environment Minesweeper*, arsitektur *agent* DQN dan CAE-DQN, konfigurasi pelatihan, skema evaluasi, serta metode analisis kuantitatif.

4. Bab IV – Hasil dan Analisis

Bab ini menyajikan hasil eksperimen beserta analisis kuantitatif dan kualitatif. Pembahasan mencakup evaluasi performa akhir, efisiensi sampel, kualitas dan adaptivitas eksplorasi, stabilitas pelatihan, serta interpretasi temuan.

5. Bab V – Kesimpulan dan Saran

Bab ini merangkum kesimpulan utama serta memberikan saran untuk pengembangan dan penelitian selanjutnya.

BAB 2

TINJAUAN PUSTAKA

Bab ini menyajikan landasan teoretis yang mendasari penelitian ini, dimulai dengan pengenalan konsep dasar *Reinforcement Learning* serta arsitektur DQN sebagai algoritma *value-based* yang menjadi fondasi eksperimen. Pembahasan selanjutnya difokuskan pada permasalahan eksplorasi dalam DQN, khususnya keterbatasan strategi ϵ -greedy pada *environment* dengan *reward* jarang dan penalti terminal seperti permainan *Minesweeper*. Karakteristik *Minesweeper* sebagai POMDP kemudian diuraikan untuk memperjelas tantangan yang dihadapi. Tinjauan terhadap studi terdahulu disajikan untuk mengidentifikasi kesenjangan penelitian yang memotivasi penerapan CAE. Pada bagian akhir, bab ini menguraikan secara rinci mekanisme eksplorasi berbasis ketidakpastian laten pada CAE serta kontribusi penelitian yang dihasilkan dari integrasi metode tersebut ke dalam kerangka DQN.

2.1 Dasar Reinforcement Learning

RL adalah paradigma *Machine Learning* (ML) di mana sebuah *agent* belajar mengambil keputusan melalui interaksi berulang dengan *environment* [7]. Pada setiap langkah waktu diskret t , *agent* mengamati *state* $s_t \in \mathcal{S}$, memilih *action* $a_t \in \mathcal{A}$ berdasarkan *policy* π , dan menerima umpan balik berupa *reward* skalar r_{t+1} serta transisi ke *state* berikutnya s_{t+1} . Tujuan *agent* adalah memaksimalkan akumulasi *reward* jangka panjang yang diformalkan sebagai *expected discounted return* (2.1), di mana G_t menyatakan *return* yang diharapkan pada waktu t , $\mathbb{E}[\cdot]$ adalah operator ekspektasi terhadap dinamika *environment* dan *policy*, r_{t+k+1} merupakan *reward* yang diterima pada langkah ke- $(t+k+1)$, dan $\gamma \in [0, 1)$ adalah *discount factor* yang mengatur tingkat kepentingan *reward* di masa depan, relatif terhadap *reward* saat ini.

$$G_t = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \right] \quad (2.1)$$

Formulasi ini bersifat umum dan dapat diterapkan pada berbagai *environment*, terma-

suk permainan berbasis logika seperti *Minesweeper*. Dalam konteks *Minesweeper*, *state* merepresentasikan konfigurasi papan yang teramati, *action* adalah pemilihan petak untuk dibuka, dan *reward* diberikan berdasarkan hasil *action*, misalnya membuka petak aman, memenangkan permainan, atau mengenai ranjau. Namun, kompleksitas representasi *state* pada *Minesweeper*, terutama dalam bentuk observasi parsial dan struktur spasial papan, membuat pendekatan tabular menjadi tidak praktis. Oleh karena itu, diperlukan metode yang mampu melakukan generalisasi dari pengalaman terbatas.

2.1.1 Arsitektur Deep Q-Network (DQN)

DQN yang diperkenalkan oleh Mnih et al. (2015) merupakan algoritma *value-based* yang menggabungkan *Q-learning* dengan jaringan saraf dalam untuk menangani *state space* berdimensi tinggi. DQN mengaproksimasi *action-value function* optimal $Q^*(s, a)$ menggunakan jaringan saraf berparameter θ , yang dinotasikan sebagai $Q(s, a; \theta)$. Pendekatan ini memungkinkan generalisasi lintas *state* yang belum pernah diamati. Stabilitas dan kinerja DQN didukung oleh tiga komponen utama:

1. Aproksimasi Fungsi dengan Jaringan Saraf

Jaringan saraf, khususnya CNN, memetakan representasi *state* mentah ke estimasi *Q-value* untuk seluruh *action* yang tersedia. Kemampuan generalisasi CNN memungkinkan *agent* menangani *state space* yang besar tanpa perlu mengunjungi setiap konfigurasi secara eksplisit.

2. *Experience Replay*

Transisi pengalaman $(s_t, a_t, r_{t+1}, s_{t+1})$ disimpan dalam *replay buffer* dan diambil secara acak selama pelatihan. Mekanisme ini memutus korelasi temporal antar sampel dan memungkinkan penggunaan kembali pengalaman secara efisien, sehingga meningkatkan stabilitas dan efisiensi pembelajaran.

3. *Target Network*

Jaringan target $\hat{Q}$ dengan parameter θ^- digunakan untuk menghitung nilai target selama proses pembaruan. Parameter θ^- diperbarui secara periodik dengan menyalin parameter jaringan utama θ dan dipertahankan tetap di antara pembaruan. Strategi

ini mengurangi efek *moving target* yang dapat menyebabkan osilasi atau divergensi selama pelatihan.

Ketiga komponen tersebut bekerja secara sinergis untuk memastikan stabilitas pembelajaran. Secara formal, pembaruan parameter jaringan dilakukan dengan meminimalkan *loss function* (2.3), yang berupa *Mean Squared Error* (MSE) antara prediksi $Q(s_t, a_t; \theta)$ dan nilai target y_t (2.2). Target ini dihitung menggunakan estimasi *target network*, di mana r_{t+1} adalah *reward* yang diterima setelah mengambil *action* a_t , $\gamma \in [0, 1)$ adalah *discount factor*, s_{t+1} adalah *state* berikutnya, a' merepresentasikan seluruh kemungkinan *action*, dan θ^- adalah parameter dari *target network* yang diperbarui secara periodik.

$$y_t = r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a'; \theta^-) \quad (2.2)$$

Sehingga, *loss function* yang diminimalkan dapat dirumuskan sebagai (2.3), di mana $\mathbb{E}[\cdot]$ menyatakan ekspektasi terhadap distribusi transisi yang diambil dari *replay buffer* $\mathcal{D}$, dan $(s_t, a_t, r_{t+1}, s_{t+1})$ adalah sampel pengalaman yang digunakan dalam proses pembelajaran.

$$L(\theta) = \mathbb{E}(s_t, a_t, r_{t+1}, s_{t+1}) \sim \mathcal{D} \left[(y_t - Q(s_t, a_t; \theta))^2 \right] \quad (2.3)$$

Dengan arsitektur dan mekanisme pembelajaran tersebut, DQN menjadi kandidat yang kuat sebagai dasar evaluasi dalam penelitian ini.

2.1.2 DQN sebagai Baseline Penelitian

Pemilihan DQN sebagai algoritma dasar (*baseline*) dalam penelitian ini didasarkan pada dua pertimbangan utama yang selaras dengan karakteristik *Minesweeper* dan tujuan eksperimen. Pertama, DQN secara fundamental dirancang untuk *action space* diskret, di mana pada setiap langkah *agent* memilih satu *action* dari himpunan terbatas. Algoritma *value-based* seperti DQN secara alami memodelkan permasalahan ini dengan mengestimasi nilai $Q(s, a)$ untuk setiap pasangan *state-action*, kemudian memilih *action* dengan nilai tertinggi.

Kedua, arsitektur DQN yang modular memungkinkan isolasi yang jelas terhadap efek dari strategi eksplorasi yang diuji. Dalam penelitian ini, seluruh komponen DQN,

meliputi arsitektur CNN, mekanisme *experience replay*, *target network*, *optimizer*, serta seluruh *hyperparameter* pelatihan, dipertahankan identik antara konfigurasi *baseline* dan konfigurasi eksperimen. Perbedaan terletak pada mekanisme pemilihan *action* selama fase eksplorasi. *Baseline* menggunakan strategi ϵ -greedy konvensional, sedangkan konfigurasi eksperimen menggantinya dengan CAE. Dengan mengisolasi variabel tunggal ini, setiap perbedaan dalam kinerja pelatihan, baik dari segi efisiensi sampel, kecepatan konvergensi, maupun *win rate* akhir, dapat diatribusikan secara langsung pada efektivitas mekanisme eksplorasi CAE. Pendekatan ini memastikan bahwa perbandingan yang dilakukan bersifat adil dan valid secara eksperimental.

2.2 Masalah Eksplorasi pada DQN

Keberhasilan algoritma RL tidak hanya ditentukan oleh akurasi estimasi nilai *action*, tetapi juga oleh kemampuannya menyeimbangkan eksplorasi (*exploration*) dan eksploitasi (*exploitation*) [7]. Eksploitasi merujuk pada kecenderungan *agent* untuk memilih *action* yang saat ini diperkirakan memberikan *reward* tertinggi, sedangkan eksplorasi bertujuan memperoleh informasi baru melalui pemilihan *action* yang belum pasti atau suboptimal. Ketidakseimbangan antara keduanya dapat berdampak signifikan terhadap kualitas pembelajaran. Eksploitasi berlebihan berisiko menyebabkan konvergensi pada *policy* suboptimal, sementara eksplorasi berlebihan menghambat konvergensi dan menurunkan efisiensi sampel.

$$a_t = \begin{cases} \text{action acak dari } \mathcal{A}, & \text{dengan probabilitas } \epsilon \\ \arg \max_{a \in \mathcal{A}} Q(s_t, a; \theta), & \text{dengan probabilitas } 1 - \epsilon \end{cases} \quad (2.4)$$

Dalam arsitektur DQN standar, keseimbangan ini umumnya dikelola melalui strategi ϵ -greedy [2, 8]. Berdasarkan mekanisme ini, *agent* memilih *action* secara acak dengan probabilitas $\epsilon \in [0, 1]$, dan memilih *action greedy* $a_t = \arg \max_a Q(s_t, a; \theta)$ dengan probabilitas $1 - \epsilon$. Nilai ϵ biasanya diturunkan secara bertahap (*decay*) dari nilai awal yang tinggi menuju nilai minimum yang kecil untuk mendorong transisi dari eksplorasi ke eksploitasi. Meskipun sederhana dan efektif pada berbagai *benchmark* seperti permainan *Atari* [2], pendekatan ini memiliki keterbatasan mendasar pada *environment* dengan *re-*

ward jarang dan penalti terminal yang fatal.

Keterbatasan tersebut menjadi sangat nyata pada permainan *Minesweeper*. Eksplorasi acak yang diberikan oleh ϵ -greedy tidak membedakan antara *action* yang berpotensi informatif dan yang bersifat destruktif. Setiap *action* yang salah, yakni membuka petak berisi ranjau, langsung mengakhiri episode. Akibatnya, pada fase awal pelatihan ketika ϵ masih tinggi, *agent* cenderung memilih *action* secara acak dan mengalami terminasi dini sebelum sempat mengamati konsekuensi jangka panjang atau membangun representasi yang memadai terhadap struktur papan. Di sisi lain, karena *reward* positif hanya diperoleh setelah rangkaian *action* aman yang cukup panjang, *agent* jarang menerima sinyal pembelajaran yang informatif pada tahap awal. Kombinasi antara terminasi dini dan *reward* jarang ini menyebabkan ϵ -greedy menjadi sangat tidak efisien secara sampel, karena sebagian besar episode berakhir tanpa kontribusi berarti terhadap pembaruan parameter jaringan [9]. Dengan kata lain, eksplorasi acak dalam konteks ini tidak hanya tidak efisien, tetapi juga secara sistematis menghambat proses pembelajaran.

Keterbatasan tersebut menunjukkan perlunya strategi eksplorasi yang lebih terarah (*directed exploration*), yang tidak hanya mengeksplorasi secara acak tetapi juga mempertimbangkan tingkat ketidakpastian dalam estimasi *value function*. Namun, sebelum menguraikan solusi tersebut, bagian selanjutnya akan membedah secara khusus karakteristik permainan *Minesweeper*, sebuah *environment* yang secara inheren mengekspos kelemahan mendasar dari ϵ -greedy.

2.3 Minesweeper: Tantangan Logika dan Observasi Parsial

Minesweeper merupakan permainan teka-teki berbasis logika yang tergolong dalam kelas *NP-complete* [10, 11], golongan ini mengimplikasikan bahwa tidak terdapat algoritma deterministik yang mampu menyelesaikan seluruh konfigurasi papan secara efisien dalam waktu polinomial. Implikasi ini menempatkan *Minesweeper* bukan sekadar sebagai permainan rekreasi, melainkan sebagai *environment* yang menantang untuk mengevaluasi kemampuan sistem kecerdasan buatan dalam menghadapi kompleksitas komputasi dan ketidakpastian.

Dalam perspektif RL, tantangan tersebut terutama bersumber dari dua karakteristik fundamental. Pertama, *Minesweeper* merupakan *environment* dengan observasi parsial.

Agent tidak memiliki akses terhadap *state* papan secara penuh, informasi hanya diperoleh secara bertahap melalui interaksi dengan *environment*. Kondisi ini secara formal memposisikan *Minesweeper* sebagai POMDP [3, 12], di mana keputusan harus diambil berdasarkan observasi yang tidak lengkap dan umumnya ambigu. Akibatnya, *agent* dituntut untuk mempertahankan dan memperbarui *belief* terhadap kemungkinan *state* yang mendasari observasi tersebut. Berbeda dengan *environment* yang sepenuhnya terobservasi seperti permainan *Atari* [2], proses pengambilan keputusan pada *Minesweeper* melibatkan penalaran di bawah ketidakpastian.

Kedua, struktur *reward* pada *Minesweeper* bersifat sangat jarang (*sparse reward*) dan disertai penalti terminal yang fatal. *Agent* hanya menerima sinyal *reward* yang terbatas sepanjang episode, sementara satu *action* yang keliru langsung mengakhiri permainan. Konsekuensinya, proses pembelajaran menjadi sangat bergantung pada kualitas eksplorasi, khususnya pada fase awal ketika *agent* belum memiliki representasi yang memadai terhadap *environment*. Dalam kondisi ini, strategi eksplorasi sederhana seperti ϵ -greedy cenderung tidak efisien, *action* acak memiliki probabilitas tinggi untuk menghasilkan terminasi dini sebelum *agent* memperoleh pengalaman yang cukup informatif.

Kombinasi antara observasi parsial, *sparse reward*, dan risiko terminasi instan menjadikan *Minesweeper* sebagai *environment* yang sangat menuntut strategi eksplorasi yang terarah dan efisien. Karakteristik ini secara langsung memotivasi pengembangan pendekatan eksplorasi yang lebih adaptif dalam literatur RL. Oleh karena itu, untuk memposisikan kontribusi penelitian ini secara tepat, bagian berikutnya akan mengkaji studi-studi terdahulu yang relevan, baik dalam konteks penerapan RL pada *Minesweeper* maupun dalam pengembangan strategi eksplorasi berbasis ketidakpastian.

2.4 Studi Terdahulu dan Posisi Penelitian

Bagian ini mengkaji penelitian-penelitian terdahulu yang relevan untuk memposisikan kontribusi penelitian secara jelas dalam konteks RL. Pembahasan difokuskan pada dua garis utama, yaitu penerapan RL pada permainan *Minesweeper* dan perkembangan strategi eksplorasi berbasis ketidakpastian. Bagian ini secara sistematis mengidentifikasi keterbatasan pendekatan yang ada, khususnya terkait skalabilitas representasi dan efisiensi sampel. Analisis tersebut kemudian digunakan untuk merumuskan kesenjangan

penelitian yang menjadi dasar motivasi dan arah utama studi ini.

2.4.1 RL untuk Permainan Minesweeper

Salah satu studi awal yang mengeksplorasi penerapan RL pada *Minesweeper* dilakukan oleh Gardea et al. (2015). Studi tersebut mengimplementasikan *tabular Q-learning* pada berbagai ukuran papan dan menunjukkan bahwa *agent* mampu mencapai *win rate* sekitar 70% pada papan kecil (4×4 dengan tiga ranjau). Namun, performa menurun drastis hingga di bawah 5% ketika ukuran papan diperbesar menjadi 5×5 dan 6×6 [4]. Kegagalan ini diatribusikan pada ledakan *state space* yang bersifat eksponensial, sehingga *agent* tidak mampu mengunjungi dan mempelajari setiap konfigurasi secara memadai. Temuan ini menegaskan keterbatasan pendekatan tabular dan secara eksplisit memotivasi penggunaan DQN dan CNN sebagai pendekatan yang lebih skalabel melalui *function approximation*.

Menindaklanjuti arah tersebut, penelitian Wang & Lei (2025) mengimplementasikan DQN berbasis CNN pada konfigurasi papan 6×6 dengan 4 ranjau. Model yang diusulkan mampu mencapai *win rate* sebesar 93.3% [9], menunjukkan bahwa pendekatan *deep RL* efektif dalam menangani kompleksitas representasi *state*. Namun, performa tersebut dicapai setelah sekitar 1.5 juta *training steps* untuk mencapai konvergensi, mengindikasikan bahwa strategi eksplorasi ϵ -greedy masih menyisakan ruang signifikan untuk peningkatan efisiensi sampel. Dengan demikian, meskipun DQN berhasil dari sisi performa akhir, efisiensi sampel masih menjadi *bottleneck* utama. Wang & Lei (2025) secara eksplisit menyatakan, “*there is still room for improvement in sample efficiency*”, mengindikasikan bahwa strategi eksplorasi yang lebih efektif masih dibutuhkan.

2.4.2 Strategi Eksplorasi Berbasis Ketidakpastian

Keterbatasan eksplorasi acak seperti ϵ -greedy telah mendorong berkembangnya pendekatan eksplorasi berbasis ketidakpastian, di mana *agent* secara aktif diarahkan untuk mengunjungi *state* yang kurang dipahami. Pendekatan ini berakar pada teori *Multi-Armed Bandit* (MAB), khususnya algoritma UCB, yang memberikan jaminan teoretis berupa *sub-linear regret* melalui keseimbangan eksplorasi dan eksploitasi [1, 13]. Dalam konteks RL, prinsip ini diimplementasikan dengan menambahkan *exploration bonus* berbasis

ketidakpastian pada estimasi nilai, sehingga *agent* tidak hanya mengejar *reward* ekstrinsik, tetapi juga mengurangi ketidaktauhan terhadap *environment*.

Salah satu pendekatan *state-of-the-art* dalam kategori ini adalah CAE yang diusulkan oleh Li (2025). Berbeda dari metode intrinsik lain yang umumnya menambah modul atau parameter baru, CAE mempertahankan kesederhanaan dengan memanfaatkan representasi laten dari jaringan *critic* yang sudah ada. Secara teknis, fitur laten $\phi(s)$ diekstraksi dari lapisan *embedding*, kemudian digunakan untuk menghitung bonus ketidakpastian berbasis UCB, yaitu $\beta\sqrt{\phi^T \mathbf{A}^{-1} \phi}$, dengan $\mathbf{A}$ sebagai *Gram matrix* yang mengakumulasi kovarians fitur dari *state* yang telah dikunjungi.

Pembaruan $\mathbf{A}$ dilakukan secara efisien menggunakan formula *Sherman–Morrison*, sementara stabilitas numerik dijaga melalui normalisasi *Welford* [1]. Karakteristik kunci dari mekanisme ini adalah sifat *self-annealing*. Seiring bertambahnya pengalaman, nilai bonus secara alami menurun tanpa memerlukan penjadwalan *hyperparameter* eksplorasi. Dengan demikian, CAE secara implisit mengatur transisi dari eksplorasi ke eksploitasi secara adaptif.

Secara empiris, penelitian menunjukkan bahwa CAE secara konsisten meningkatkan efisiensi sampel dan stabilitas pelatihan pada berbagai *benchmark*, termasuk *MuJoCo* dan *MiniHack* [1]. Namun, efektivitasnya pada *environment* berbasis logika dengan penalti terminal seperti *Minesweeper* masih belum dieksplorasi, secara efektif membuka celah penelitian yang menjadi fokus utama dalam studi ini.

2.4.3 Kesenjangan dan Motivasi Penelitian

Terdapat dua kesenjangan utama yang melatarbelakangi penelitian ini. Pertama, DQN telah terbukti mampu menyelesaikan permainan *Minesweeper* 6×6 dengan 4 ranjau hingga mencapai *win rate* 93,3% [9]. Namun, pencapaian tersebut memerlukan sekitar 1,5 juta *training steps* (~ 200.000 episode), yang mengindikasikan rendahnya efisiensi sampel dari strategi eksplorasi ϵ -greedy. Keterbatasan ini secara eksplisit diakui dalam literatur, sehingga membuka kebutuhan akan mekanisme eksplorasi yang lebih efisien untuk mempercepat proses pembelajaran pada *environment* yang sama.

Kedua, CAE menawarkan pendekatan eksplorasi berbasis ketidakpastian laten yang ringan, tanpa penambahan parameter jaringan, serta memiliki sifat *self-annealing* [1].

Meskipun terbukti meningkatkan efisiensi sampel pada berbagai *benchmark* seperti *MuJoCo* dan *MiniHack*, efektivitasnya belum pernah dievaluasi pada *environment* dengan karakteristik seperti *Minesweeper*: ruang *action* diskret, observasi parsial (POMDP), struktur berbasis logika, dan penalti terminal yang fatal. Dalam konteks ini, kualitas eksplorasi menjadi faktor kritis, eksplorasi yang tidak terarah cenderung menghasilkan terminasi dini dan membatasi akumulasi pengalaman yang informatif.

Berdasarkan kedua kesenjangan tersebut, penelitian ini menginvestigasi integrasi CAE ke dalam arsitektur DQN pada *environment Minesweeper* 6×6 dengan 4 ranjau. Penelitian ini menggunakan arsitektur CNN yang lebih ringkas dibandingkan dengan penelitian Wang & Lei (2025). Oleh karena itu, tujuan evaluasi bukan untuk menandingi *win rate* absolut 93.3% [9] yang dilaporkan, melainkan untuk menilai peningkatan relatif terhadap *baseline ϵ -greedy* dalam kondisi sumber daya komputasi yang setara. Dengan demikian, fokus utama penelitian ini adalah mengevaluasi apakah CAE mampu mempercepat konvergensi, menghasilkan eksplorasi yang lebih terarah, dan mencapai kinerja akhir yang lebih baik pada arsitektur DQN yang identik dengan *training budget* yang terbatas.

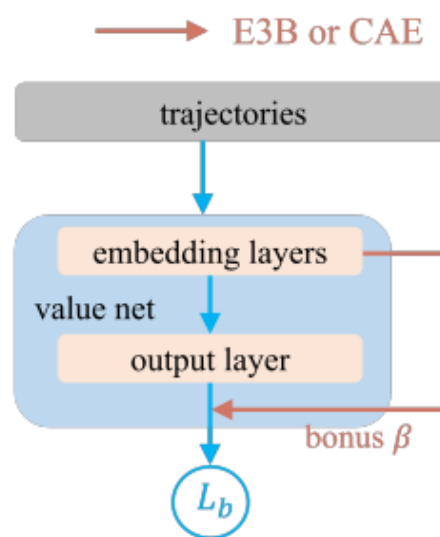
2.5 Strategi Eksplorasi Critic as Explorer (CAE)

CAE diusulkan sebagai metode eksplorasi ringan yang memanfaatkan kembali jaringan *critic* dalam arsitektur DQN [1]. Pendekatan ini bertujuan meningkatkan efisiensi sampel dengan menambahkan bonus eksplorasi berbasis ketidakpastian, tanpa memperkenalkan parameter tambahan ke dalam model. Berbeda dengan metode motivasi intrinsik konvensional yang umumnya memerlukan modul atau jaringan terpisah, CAE mempertahankan kesederhanaan implementasi sekaligus efisiensi komputasi.

Meskipun tidak secara khusus dirancang untuk *environment* dengan *reward* jarang dan penalti terminal yang fatal, sifat eksplorasi terarah yang dimiliki menjadikan CAE sebagai kandidat yang menjanjikan untuk diterapkan pada domain dengan karakteristik tersebut, seperti *Minesweeper*.

2.5.1 Prinsip Kerja CAE

Ide utama dari CAE adalah memanfaatkan representasi laten yang dihasilkan oleh jaringan *critic* untuk mengukur tingkat ketidakpastian (*uncertainty*) terhadap suatu *state* [1]. Dalam arsitektur DQN standar, jaringan *critic* memetakan *state* masukan melalui serangkaian lapisan konvolusional dan *fully connected*, sebelum menghasilkan estimasi *Q-value* untuk setiap *action*. Lapisan terakhir sebelum keluaran, umum disebut sebagai lapisan *embedding*, menghasilkan vektor fitur laten $\phi(s)$ yang merepresentasikan karakteristik esensial dari *state*. Vektor ini kemudian dimanfaatkan oleh CAE sebagai proksi untuk mengestimasi sejauh mana *state* serupa telah dieksplorasi selama proses pelatihan. Seperti yang diilustrasikan pada Gambar 2.1, *trajectory* pengalaman yang dikumpulkan oleh *agent* terlebih dahulu diproses oleh jaringan *value network*. *State* masukan dilewatkan melalui lapisan *embedding* untuk menghasilkan representasi laten $\phi(s)$, yang kemudian digunakan tidak hanya untuk menghitung estimasi *Q-value* pada *output layer*, tetapi juga untuk mengukur tingkat ketidakpastian. Secara paralel, CAE menghitung *exploration bonus* β berdasarkan distribusi fitur laten tersebut, yang mencerminkan seberapa sering representasi serupa telah diamati sebelumnya. Bonus ini kemudian diintegrasikan kembali ke dalam proses pembelajaran (melalui komponen L_b), sehingga mempengaruhi pembaruan jaringan dengan mendorong eksplorasi pada *state* yang belum banyak dikunjungi.



Gambar 2.1. Arsitektur konseptual CAE [1]

Secara konseptual, CAE menggeser paradigma eksplorasi dari mencoba secara acak, menjadi mencoba secara terarah berdasarkan ketidakpastian. Berbeda dengan ϵ -greedy yang memperlakukan seluruh *action* secara setara selama fase eksplorasi, CAE memberikan preferensi pada *action* yang mengarah ke *state* dengan tingkat ketidakpastian tinggi, yakni *state* yang representasi latennya belum banyak terakumulasi dalam pengalaman *agent*. Dengan demikian, *agent* didorong untuk secara aktif mengeksplorasi wilayah *state space* yang belum terpetakan, sehingga mempercepat perolehan informasi yang relevan untuk menyelesaikan permainan.

2.5.2 Mekanisme Bonus Ketidakpastian

Untuk mengkuantifikasi tingkat ketidakpastian suatu *state*, CAE mengadopsi formulasi UCB yang berakar pada teori MAB [13, 14]. Bonus eksplorasi $b(s)$ untuk *state* s dihitung berdasarkan vektor fitur laten $\phi(s)$ dan *Gram matrix* $\mathbf{A}$, yang mengakumulasi informasi kovarians dari *state* yang telah dikunjungi (2.5), di mana $\beta > 0$ adalah koefisien eksplorasi yang mengontrol kontribusi bonus, relatif terhadap *reward* ekstrinsik.

$$b(s) = \beta \sqrt{\phi(s)^T \mathbf{A}^{-1} \phi(s)} \quad (2.5)$$

Gram matrix $\mathbf{A}$ diinisialisasi sebagai $\mathbf{A} = \lambda \mathbf{I}$, dengan λ sebagai parameter regularisasi *ridge*, dan diperbarui secara inkremental setiap kali *agent* mengunjungi *state* baru menggunakan formula *Sherman–Morrison* untuk menjaga efisiensi komputasi [5]. Nilai bonus dinormalisasi secara *running* menggunakan algoritma *Welford* [6] untuk menjaga stabilitas numerik. Bonus $b(s)$ kemudian diintegrasikan ke dalam estimasi *Q-value*, sehingga *policy* pemilihan *action* didasarkan pada nilai teraugmentasi (2.6).

$$Q^{\text{aug}}(s, a) = Q(s, a; \theta) + b(s) \quad (2.6)$$

Dengan demikian, *agent* tidak hanya mengoptimalkan *reward* ekstrinsik, tetapi juga secara eksplisit didorong untuk mengurangi ketidakpastian terhadap *environment*. Karakteristik penting dari mekanisme ini adalah sifat *self-annealing*. Pada tahap awal pelatihan, *Gram matrix* $\mathbf{A}$ masih didominasi oleh komponen regularisasi $\lambda \mathbf{I}$, sehingga $\mathbf{A}^{-1}$ bernilai relatif besar dan menghasilkan bonus eksplorasi yang tinggi. Seiring bertambahnya

pengalaman, fitur laten dari *state* yang sering dikunjungi terakumulasi dalam $\mathbf{A}$, menyebabkan $\mathbf{A}^{-1}$ mengecil secara gradual dan menurunkan nilai bonus. Proses ini terjadi secara intrinsik tanpa memerlukan penjadwalan *decay hyperparameter*, memungkinkan transisi yang mulus dari eksplorasi ke eksploitasi. Sifat ini menjadi keunggulan utama CAE dibandingkan ϵ -greedy, yang bergantung pada eksperimentasi *trial-and-error* jadwal *decay* ϵ secara manual.

2.6 Kontribusi Penelitian

Penelitian ini memberikan tiga kontribusi utama terhadap literatur di bidang RL, khususnya dalam konteks eksplorasi pada *environment* dengan karakteristik observasi parsial dan penalti terminal.

Pertama, penelitian ini menyajikan evaluasi empiris pertama terhadap integrasi CAE [1] ke dalam arsitektur DQN pada domain permainan logika diskret *Minesweeper*. Berbeda dengan *benchmark* sebelumnya yang didominasi oleh tugas kontrol kontinu (seperti *MuJoCo*) dan navigasi (seperti *MiniHack*), *Minesweeper* menghadirkan tantangan unik berupa *state space* diskret, kebutuhan inferensi logis, serta konsekuensi fatal dari setiap *action* yang salah. Dengan mengevaluasi CAE pada domain ini, penelitian ini memperluas pemahaman mengenai generalitas sekaligus keterbatasan metode eksplorasi berbasis ketidakpastian laten.

Kedua, penelitian ini menyajikan analisis kuantitatif yang komprehensif terhadap efisiensi sampel dan kualitas eksplorasi CAE dibandingkan dengan *baseline* DQN berbasis ϵ -greedy. Analisis ini tidak hanya berfokus pada metrik konvensional seperti *win rate* akhir, tetapi juga mencakup metrik turunan seperti *Area Under Curve* (AUC), jumlah episode untuk mencapai ambang performa tertentu, serta *progress per click* sebagai indikator keterarahan eksplorasi. Pendekatan multi-metrik ini memungkinkan karakterisasi yang lebih mendalam terhadap dinamika pembelajaran, bukan sekadar hasil akhir.

Ketiga, penelitian ini mendokumentasikan dan memvalidasi secara empiris mekanisme *self-annealing* dari bonus eksplorasi CAE pada *environment* dengan terminasi dini. Melalui analisis evolusi nilai bonus sepanjang pelatihan, penelitian ini menunjukkan bahwa penurunan intensitas eksplorasi terjadi secara alami seiring dengan akumulasi pengalaman dalam *Gram matrix*, tanpa memerlukan penjadwalan *decay hyperparameter* se-

cara manual. Temuan ini menegaskan bahwa sifat adaptif CAE yang dilaporkan pada *benchmark* sebelumnya tetap berlaku pada domain dengan *reward* jarang dan penalti fatal, sekaligus memberikan wawasan baru mengenai perilaku *self-annealing* dalam kondisi dengan panjang lintasan eksplorasi yang terbatas akibat terminasi dini.

Secara keseluruhan, ketiga kontribusi ini diharapkan dapat mengisi kesenjangan yang teridentifikasi dalam literatur serta menyediakan landasan empiris bagi pengembangan strategi eksplorasi yang lebih efisien pada *environment* diskret berbasis logika dengan konsekuensi ireversibel.

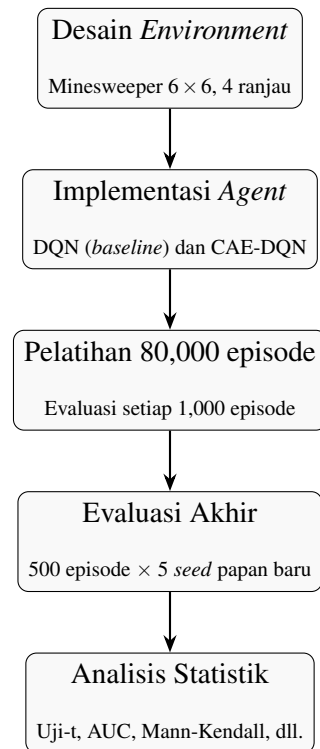
BAB 3

PERANCANGAN DAN IMPLEMENTASI SISTEM

Bab ini menguraikan perancangan dan implementasi sistem yang digunakan dalam penelitian, mencakup konfigurasi *environment Minesweeper*, arsitektur *agent* DQN dan CAE-DQN, prosedur pelatihan, skema evaluasi, serta metode analisis kuantitatif yang diterapkan. Seluruh komponen dirancang untuk memastikan perbandingan yang adil antara *baseline* dan metode yang diusulkan, satu-satunya variabel pembeda terletak pada mekanisme eksplorasi.

3.1 Prosedur Pelatihan

Secara garis besar, penelitian mengikuti alur eksperimen yang terdiri dari lima tahapan utama: perancangan *environment*, implementasi *agent*, pelatihan dengan evaluasi berkala, pengujian akhir, dan analisis statistik.



Gambar 3.1. Diagram alur prosedur pelatihan secara umum

Tahap pertama adalah perancangan *environment Minesweeper* dengan papan 6×6 dan 4 ranjau, yang mengimplementasikan antarmuka *Gym-like* serta representasi *state* berbasis *one-hot encoding*. Tahap kedua berupa implementasi dua *agent* dengan arsitektur CNN yang identik, yaitu DQN dengan eksplorasi ϵ -greedy sebagai *baseline* dan CAE-DQN dengan bonus ketidakpastian UCB sebagai metode yang diusulkan. Selanjutnya, kedua *agent* dilatih selama 80,000 episode dengan *seed* tetap. Setiap 1,000 episode, performa *agent* dipantau melalui rata-rata *win rate* dari 1,000 episode pelatihan terakhir. Model dengan *win rate* pelatihan tertinggi disimpan untuk digunakan pada evaluasi akhir. Model terbaik dari masing-masing *agent* diuji pada 5 *seed* papan yang berbeda (masing-masing 500 episode) untuk mengukur kemampuan generalisasi (*policy greedy*). Terakhir, data yang terkumpul dianalisis menggunakan serangkaian uji statistik untuk menjawab rumusan masalah penelitian secara kuantitatif.

3.2 Desain Environment

Environment Minesweeper diimplementasikan dalam kelas `MinesweeperEnv` pada file `minesweeper_env.py`. Kelas ini dirancang mengikuti antarmuka standar *Gym-like* yang umum digunakan dalam penelitian *Reinforcement Learning*, sehingga memudahkan integrasi dengan algoritma *agent* yang dikembangkan.

3.2.1 Konfigurasi Papan dan Ranjau

Environment menggunakan papan berukuran tetap 6×6 dengan jumlah ranjau sebanyak 4 buah, merujuk pada konfigurasi *Minesweeper* Wang & Lei (2025). Konfigurasi ini dipilih karena cukup kompleks untuk mengekspos kelemahan strategi eksplorasi acak, namun tetap memungkinkan pelatihan dilakukan dalam batasan komputasi yang wajar. Penempatan ranjau dilakukan secara acak menggunakan *pseudo-random number generator* yang dapat di-*seed* untuk menjamin reproduisibilitas eksperimen.

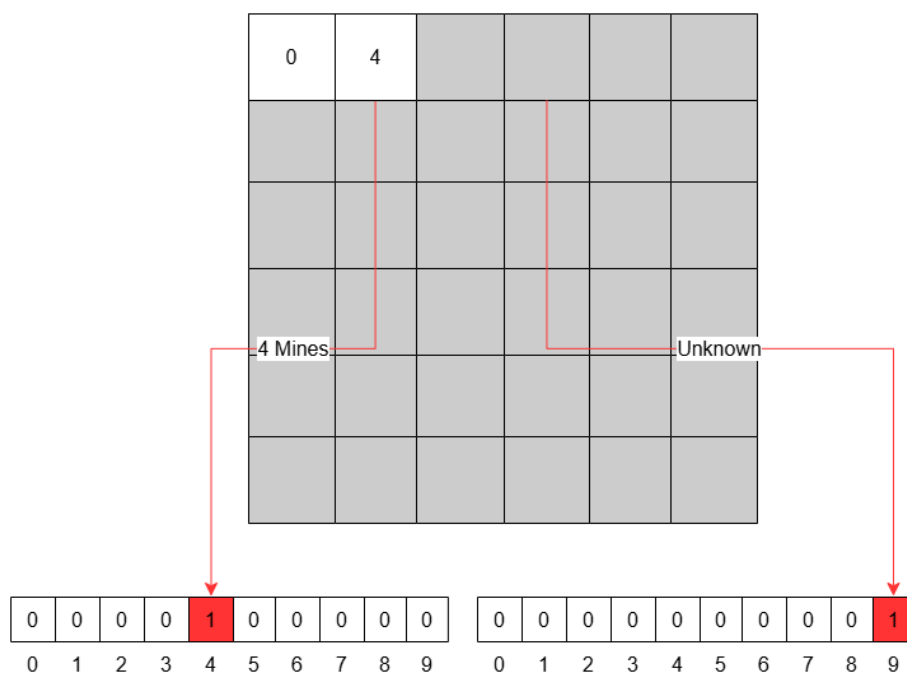
3.2.2 Representasi State: One-Hot Encoding

State yang diamati oleh *agent* direpresentasikan sebagai tensor tiga dimensi berukuran $6 \times 6 \times 10$. Secara intuitif, representasi ini dapat dibayangkan sebagai tumpukan 10 lem-

bar peta transparan, masing-masing berukuran 6×6 petak. Setiap petak pada papan di-representasikan oleh sebuah vektor sepanjang 10 elemen yang hanya berisi nilai 0 dan 1, mengikuti skema *one-hot encoding*. Tepat satu elemen bernilai 1 (aktif), sementara yang lain bernilai 0, seperti yang diilustrasikan pada Gambar 3.2. Posisi elemen yang aktif inilah yang menunjukkan kondisi petak tersebut. Sepuluh kanal (lembar transparan) tersebut memiliki peran sebagai berikut:

- **Kanal 0 sampai 8:** Merepresentasikan petak yang telah terbuka dengan nilai angka 0 hingga 8. Sebagai contoh, jika sebuah petak terbuka dan bernilai 3, maka hanya kanal ke-3 yang bernilai 1, sedangkan kanal lainnya (0, 1, 2, 4, 5, 6, 7, 8, 9) bernilai 0.
- **Kanal 9:** Merepresentasikan petak yang **belum terbuka** (*unknown*). Jika petak masih tertutup, maka kanal ke-9 bernilai 1, sementara kanal lainnya bernilai 0.

Perlu diperhatikan bahwa tidak terdapat kanal khusus untuk ranjau. *Agent* tidak pernah mengamati ranjau secara langsung selama episode berlangsung. Jika *agent* memilih petak yang memiliki ranjau, episode akan langsung berakhir dan *agent* hanya menerima penalti, tanpa memperoleh representasi eksplisit mengenai lokasi ranjau tersebut.



Gambar 3.2. Contoh representasi *One-Hot Encoding* dari *Minesweeper* 6×6

Dengan demikian, *agent* dipaksa untuk melakukan inferensi berdasarkan pola angka yang terungkap secara bertahap, serupa dengan cara manusia memainkan *Minesweeper*. Representasi *one-hot* ini dipilih karena mampu mempertahankan informasi spasial dan numerik secara eksplisit, serta dapat langsung diproses oleh arsitektur CNN yang digunakan sebagai model pembelajaran.

3.2.3 Struktur Reward

Fungsi *reward* dirancang untuk memberikan sinyal pembelajaran yang informatif meskipun dalam kondisi *sparse reward* alami pada permainan *Minesweeper*. Nilai *reward* untuk setiap instansi didefinisikan pada Tabel 3.1

Tabel 3.1. Struktur *Reward Environment Minesweeper*

Kejadian	Nilai <i>Reward</i>	Deskripsi
<i>Win</i>	+36	<i>Agent</i> berhasil membuka semua petak aman.
<i>Lose</i>	-36	<i>Agent</i> memilih petak berisi ranjau.
<i>Progress</i>	+1.0	Membuka setidaknya satu petak aman baru.

3.2.4 Kondisi Terminasi dan Observabilitas Parsial

Sebuah episode dinyatakan berakhir (*done*) apabila salah satu dari dua kondisi berikut terpenuhi:

1. **Kalah:** *Agent* memilih petak yang berisi ranjau.
2. **Menang:** Seluruh petak yang tidak mengandung ranjau telah berhasil dibuka, yang secara ekuivalen terjadi ketika jumlah petak yang belum terbuka sama dengan jumlah ranjau (4).

Selain kondisi terminasi tersebut, *environment* ini juga merefleksikan karakteristik POMDP melalui observabilitas parsial. *Agent* tidak memiliki akses langsung terhadap papan solusi penuh (`self.board`), melainkan hanya terhadap representasi *state* dalam bentuk citra *one-hot* (`self.state_im`) pada waktu tertentu. Informasi mengenai lokasi ranjau serta nilai pada petak yang belum terbuka tetap tersembunyi, sehingga *agent* harus

melakukan inferensi berdasarkan pola angka yang terungkap secara bertahap selama interaksi dengan *environment*.

3.2.5 Mekanisme Tambahan

Dua mekanisme tambahan diimplementasikan untuk meningkatkan stabilitas pelatihan sekaligus mencerminkan perilaku permainan standar:

- **Keamanan Klik Pertama:** Untuk menghindari terminasi instan, *environment* secara otomatis memindahkan ranjau jika *agent* memilih petak ranjau pada klik pertama, sehingga *agent* tetap dapat memulai proses penalaran.
- **Pembukaan Rekursif Nol:** Jika *agent* membuka petak bernilai 0, seluruh petak tetangganya akan terbuka secara rekursif hingga mencapai batas petak bernilai bukan 0.

Seluruh komponen *environment* dirancang deterministik untuk *seed* yang tetap, sehingga eksperimen dapat direproduksi secara konsisten dan memberikan fondasi yang beragam bagi pelatihan kedua *agent*.

Tabel 3.2. Ringkasan Konfigurasi *Environment*

Parameter	Nilai
Ukuran Papan	6×6 petak
Jumlah Ranjau	4
Dimensi <i>State</i>	$6 \times 6 \times 10$ kanal
<i>Action Space</i>	36 diskret
<i>Seed Environment</i> (Pelatihan)	2004
<i>Seed Environment</i> (Evaluasi)	1, 2, 3, 4, 5

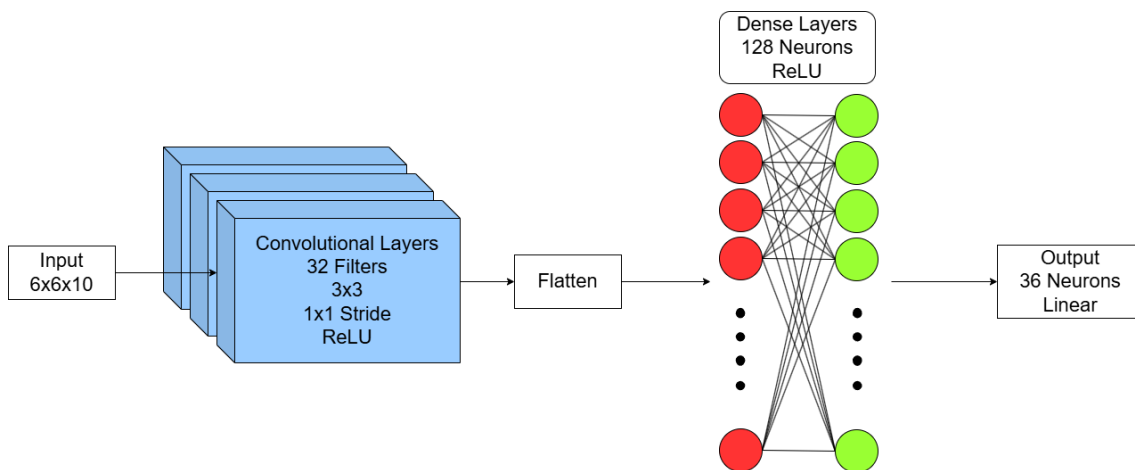
3.3 Desain Algoritma

Bagian ini menguraikan perancangan dua *agent* yang dibandingkan, DQN sebagai *base-line* dan CAE-DQN sebagai metode yang diusulkan. Keduanya dibangun menggunakan arsitektur CNN dan *hyperparameter* yang identik, sehingga strategi eksplorasi menjadi

satu-satunya variabel pembeda. Dengan demikian, perbedaan kinerja dapat diatribusikan langsung pada mekanisme eksplorasi yang digunakan.

3.3.1 DQN

Agent DQN diimplementasikan sebagai *baseline* dalam penelitian ini, mengacu pada arsitektur klasik yang diperkenalkan oleh Mnih et al. (2015). Implementasi terdiri atas dua *file* utama, yaitu `dqn_model.py` untuk mendefinisikan arsitektur jaringan dan `dqn_agent.py` untuk menangani pemilihan *action*, penyimpanan pengalaman, serta pembaruan parameter.



Gambar 3.3. Arsitektur CNN dari DQN pada Eksperimen

Model DQN dibangun menggunakan *TensorFlow/Keras* dengan arsitektur CNN yang terdiri atas tiga lapisan konvolusional dan dua lapisan *fully connected*, seperti ditunjukkan pada Gambar 3.3. Setiap lapisan konvolusional memiliki 32 *filter* berukuran 3×3 , *stride* 1×1 , *padding* same, dan aktivasi *Rectified Linear Unit* (ReLU). Setelah *flatten*, representasi diproses oleh dua lapisan *fully connected* dengan 128 neuron dan aktivasi ReLU. Lapisan keluaran terdiri atas 36 neuron—sesuai jumlah petak pada papan 6×6 —dengan aktivasi linear untuk menghasilkan estimasi $Q(s, a)$. Arsitektur ini dipilih untuk menjaga efisiensi komputasi tanpa mengorbankan kapasitas representasi.

Selama pelatihan, *agent* memilih *action* menggunakan strategi ϵ -greedy (2.4). Dengan probabilitas ϵ , *agent* memilih *action* secara acak dari himpunan petak yang masih belum terbuka, sedangkan dengan probabilitas $1 - \epsilon$, *agent* memilih *action* dengan nilai

Q tertinggi. Nilai ϵ diinisialisasi sebesar 0.95 dan meluruh secara eksponensial dengan faktor 0.99995 hingga mencapai batas minimum 0.02.

Untuk mencegah pemilihan *action* pada petak yang telah terbuka, diterapkan mekanisme *masking* dengan menetapkan nilai Q menjadi $-\infty$ pada indeks petak tersebut. Kondisi ini diidentifikasi melalui kanal ke-9 pada representasi *state*, di mana nilai 0 menunjukkan bahwa petak tidak lagi berstatus *unknown*. Dengan demikian, petak yang telah terbuka tidak akan pernah dipilih oleh operasi $\arg \max$. DQN selanjutnya menerapkan dua mekanisme utama:

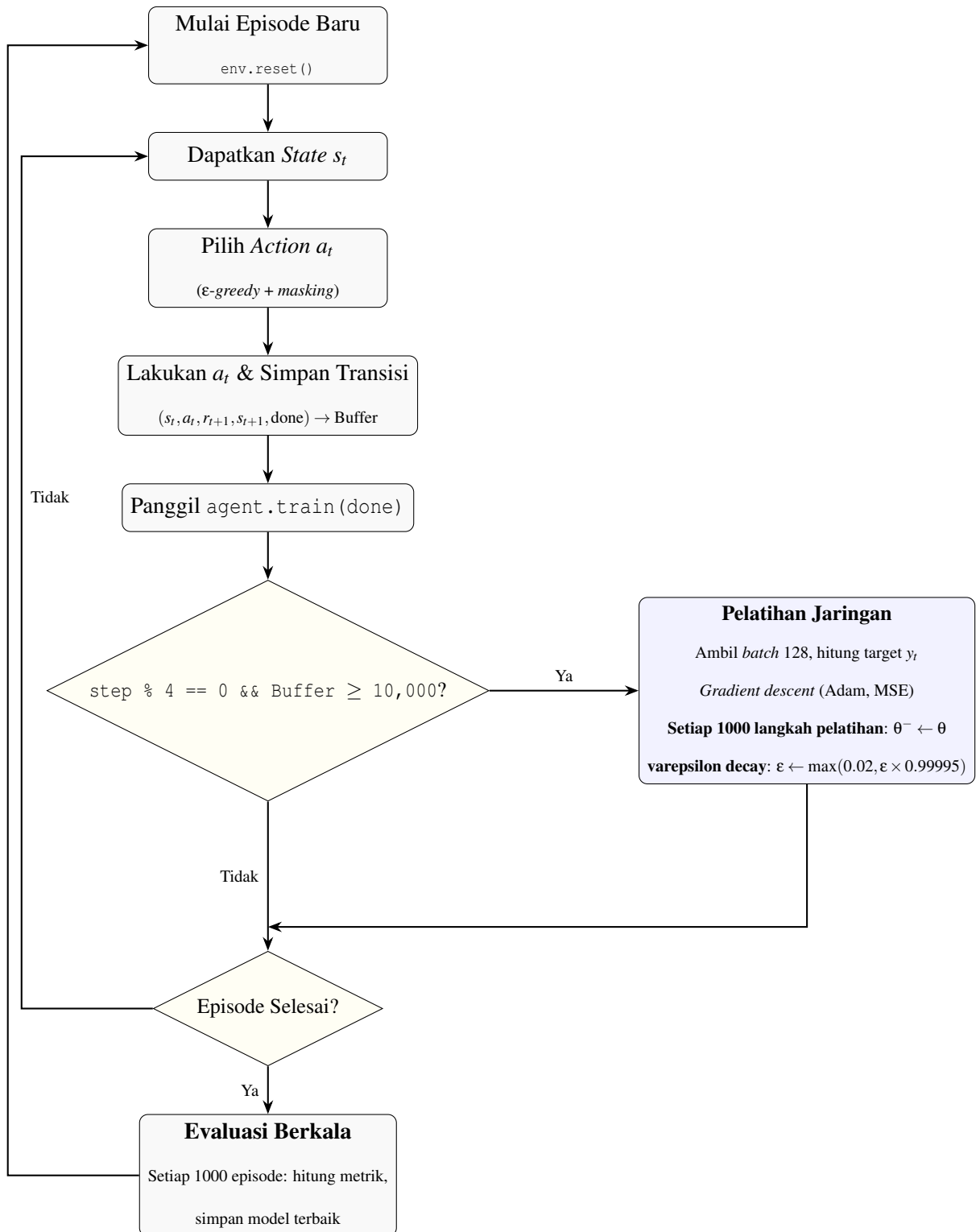
1. *Experience Replay*

Transisi $(s_t, a_t, r_{t+1}, s_{t+1}, done)$ disimpan dalam *replay buffer* berkapasitas 300,000 sampel. Selama pelatihan, *batch* berukuran 128 diambil secara acak untuk memutus korelasi temporal dan memungkinkan penggunaan ulang pengalaman. Pelatihan dimulai setelah *buffer* terisi minimal 10,000 transisi.

2. *Target Network*

Jaringan terpisah dengan arsitektur identik digunakan sebagai *target network* (2.2) untuk menghitung nilai target y_t . Parameter θ^- diperbarui dengan menyalin parameter jaringan utama θ setiap 1,000 langkah pelatihan (1 langkah pelatihan dilakukan setiap 4 langkah interaksi dengan *environment*). Mekanisme ini menjaga stabilitas target dan mengurangi risiko osilasi.

Pembaruan parameter jaringan utama dilakukan dengan meminimalkan MSE antara prediksi $Q(s_t, a_t; \theta)$ dan target y_t menggunakan *optimizer Adaptive Moment Estimation* (ADAM) [15] dengan *Learning Rate* (LR) 0.001 dan parameter $\epsilon = 10^{-4}$. Pembaruan dilakukan setiap 4 langkah interaksi untuk menyeimbangkan akumulasi pengalaman dan frekuensi pelatihan. Pada setiap langkah pelatihan, nilai ϵ diperbarui dan penghitung pembaruan *target network* ditingkatkan, sehingga proses pembelajaran berlangsung secara berkala meskipun interaksi dengan *environment* terjadi secara kontinu.



Gambar 3.4. Diagram alur pelatihan DQN

Diagram 3.4 mengilustrasikan alur pelatihan DQN. Setelah inialisasi, *agent* secara iteratif mengamati *state* s_t , memilih *action* a_t menggunakan strategi ϵ -greedy dengan

masking, melakukan *action*, dan menyimpan transisi ke dalam *replay_buffer*.

Fungsi `train()` dipanggil pada setiap langkah, namun pembaruan jaringan hanya dilakukan jika dua kondisi terpenuhi: (1) langkah interaksi merupakan kelipatan 4, dan (2) *replay buffer* telah berisi minimal 10,000 sampel. Jika terpenuhi, *batch* acak berukuran 128 diambil dari *replay_buffer* (kapasitas 300,000) untuk memperbarui parameter jaringan utama menggunakan *optimizer* ADAM dan *loss* MSE. Setelah pembaruan, nilai ϵ diperbarui sesuai jadwal *decay*.

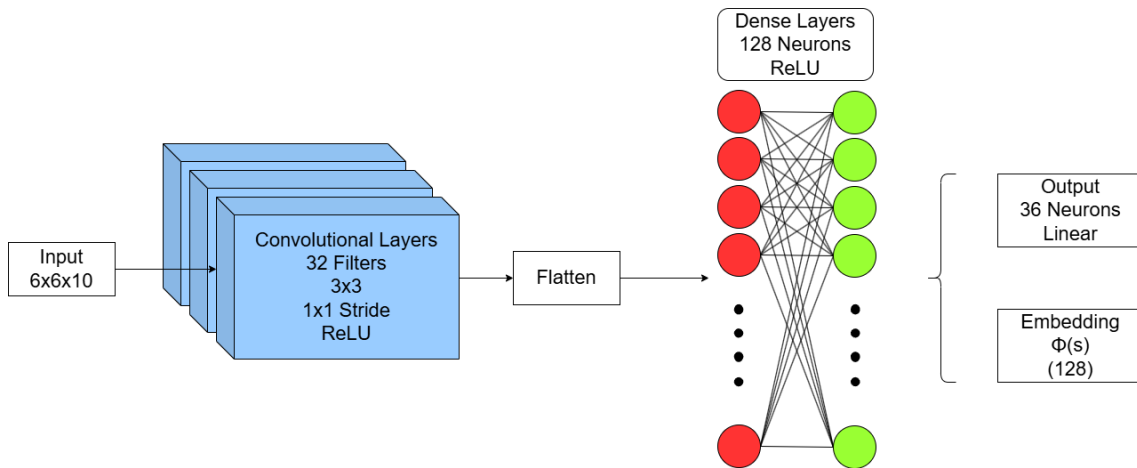
Target network ($\theta^- \leftarrow \theta$) diperbarui secara periodik setiap 1,000 langkah pelatihan. Siklus ini berlanjut hingga episode berakhir, dengan evaluasi berkala setiap 1,000 episode untuk memantau kinerja dan menyimpan model terbaik.

Tabel 3.3. Ringkasan *Hyperparameter* dan Konfigurasi DQN

Parameter	Nilai / Deskripsi
<i>Arsitektur Jaringan</i>	
Arsitektur CNN	3 lapis konvolusi (32 <i>filter</i> , 3×3 , ReLU)
<i>Fully Connected</i>	2 lapis (128 neuron, ReLU)
Dimensi Masukan / Keluaran	$6 \times 6 \times 10 / 36$ (<i>linear</i>)
<i>Optimasi & Pelatihan</i>	
<i>Optimizer</i>	Adam ($\alpha = 0.001$, $\epsilon = 10^{-4}$)
<i>Loss Function</i>	MSE
<i>Batch Size / γ</i>	128 / 0.9
Frekuensi Pelatihan	Setiap 4 langkah interaksi
<i>Experience Replay & Target Network</i>	
Kapasitas <i>Buffer</i>	300,000 (mulai melatih pada 10,000)
<i>Target Update</i>	Setiap 1,000 langkah pelatihan
<i>Eksplorasi ϵ-greedy</i>	
ϵ Jadwal	$0.95 \rightarrow 0.02$ (<i>exponential decay</i>)
<i>Decay Factor</i>	0.99995 per langkah pelatihan
<i>Konfigurasi Umum</i>	
Total Episode	80,000
Evaluasi Berkala	Setiap 1,000 episode
<i>Random Seed</i>	2004 (<i>Env</i>), 123 (<i>Model</i>)

3.3.2 CAE-DQN

Agent CAE-DQN mengintegrasikan mekanisme eksplorasi CAE ke dalam kerangka DQN yang telah dijelaskan sebelumnya. Implementasi terdiri atas dua *file* utama, yaitu `caedqn_model.py` yang mendefinisikan arsitektur jaringan dengan dua keluaran, serta `caedqn_agent.py` yang menangani logika pemilihan *action*, pembaruan *Gram matrix*, penyimpanan pengalaman, dan pembaruan parameter. Arsitektur CNN yang digunakan **identik** dengan *agent* DQN, terdiri atas tiga lapisan konvolusional dengan 32 *filter* berukuran 3×3 dan dua lapisan *fully connected* dengan 128 neuron. Perbedaan utama terletak pada pemanfaatan lapisan kedua terakhir sebagai lapisan *embedding* yang menghasilkan vektor fitur laten $\phi(s) \in \mathbb{R}^{128}$, yang merepresentasikan karakteristik *state*, seperti yang diilustrasikan oleh Gambar 3.5. Dengan demikian, model memiliki dua keluaran: nilai $Q(s, a)$ sepanjang 36 neuron dan vektor $\phi(s)$ yang digunakan khusus untuk menghitung bonus eksplorasi.

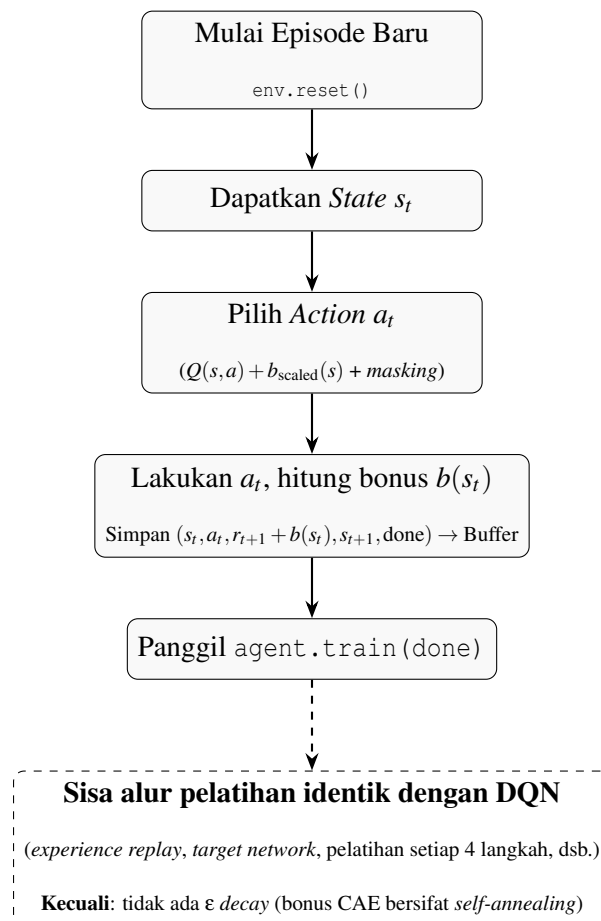


Gambar 3.5. Arsitektur CNN dari CAE-DQN pada Eksperimen

Selama pelatihan, *agent* CAE-DQN tidak lagi menggunakan strategi ϵ -greedy. Pada setiap langkah, *agent* menghitung bonus ketidakpastian $b(s)$ berdasarkan formulasi UCB pada ruang laten $\phi(s)$ (2.5), dengan $\beta = 0.1$ sebagai koefisien eksplorasi dan $\mathbf{A}$ sebagai *Gram matrix* yang mengakumulasi informasi kovarians dari fitur laten *state* yang telah dikunjungi. Matriks $\mathbf{A}$ diinisialisasi sebagai $\mathbf{A} = \lambda \mathbf{I}$ dengan $\lambda = 1.0$ sebagai regularisasi *ridge*, dan diperbarui secara inkremental bersama inversnya $\mathbf{A}^{-1}$ menggunakan formula *Sherman–Morrison* untuk menjaga efisiensi komputasi. Untuk stabilitas numerik, bonus

mentah dinormalisasi secara *running* menggunakan algoritma *Welford*, yang memperkirakan rerata dan variansi secara kumulatif.

Bonus yang telah dinormalisasi kemudian ditambahkan ke nilai $Q(s, a)$ untuk membentuk nilai teraugmentasi (2.6), yang digunakan pada fase pemilihan *action*. Selain itu, bonus yang sama juga ditambahkan ke reward ekstrinsik r_{t+1} sebelum transisi disimpan ke dalam *replay buffer*, sehingga target pembelajaran y_t mencerminkan *augmented reward*. Modifikasi ini mendorong *agent* untuk tidak hanya memaksimalkan *reward* ekstrinsik, tetapi juga secara implisit mengurangi ketidakpastian terhadap *state* yang belum banyak dieksplorasi.



Gambar 3.6. Diagram alur pelatihan CAE-DQN. Proses setelah `agent.train(done)` sama dengan DQN namun tanpa mekanisme ϵ -decay.

Proses pelatihan CAE-DQN mengikuti alur yang sama dengan DQN dalam hal *experience replay*, *target network*, frekuensi pelatihan setiap 4 langkah, serta penggunaan *optimizer* ADAM dengan LR 0.001, seperti yang ditunjukkan oleh diagram 3.6. Namun,

terdapat satu penyesuaian penting, *reward* ekstrinsik r_{t+1} ditambah dengan bonus eksplorasi $b(s_t)$ sebelum disimpan ke dalam *replay buffer* dan digunakan dalam perhitungan target y_t . Modifikasi ini mendorong *agent* untuk tidak hanya memaksimalkan *reward* ekstrinsik, tetapi juga secara implisit mengurangi ketidakpastian terhadap *state* yang belum banyak dieksplorasi.

Seiring bertambahnya pengalaman, fitur laten dari *state* yang sering dikunjungi terakumulasi dalam $\mathbf{A}$, menyebabkan $\mathbf{A}^{-1}$ mengecil secara gradual dan nilai bonus eksplorasi menurun secara alami. Sifat *self-annealing* ini menghilangkan kebutuhan akan penjadwalan *decay* manual sebagaimana pada ϵ -greedy.

Tabel 3.4. Ringkasan *Hyperparameter* dan Konfigurasi CAE-DQN

Parameter	Nilai / Deskripsi
<i>Arsitektur Jaringan</i>	
Arsitektur CNN	3 lapis konvolusi (32 <i>filter</i> , 3×3 , ReLU)
<i>Fully Connected</i>	2 lapis (128 neuron, ReLU)
Dimensi Masukan / Keluaran	$6 \times 6 \times 10 / 36$ (linear) + 128 (<i>embedding</i>)
<i>Optimasi & Pelatihan</i>	
Sama dengan Konfigurasi DQN 3.3	
<i>Experience Replay & Target Network</i>	
Sama dengan Konfigurasi DQN 3.3	
<i>Eksplorasi CAE</i>	
Koefisien Eksplorasi β	0.1
Regularisasi <i>Ridge</i> λ	1.0
Pembaruan <i>Gram Matrix</i>	<i>Sherman–Morrison</i> (setiap langkah)
Normalisasi Bonus	<i>Welford</i> (<i>running mean & variance</i>)
Rekalkulasi $\mathbf{A}^{-1}$ Penuh	Setiap 10,000 langkah
<i>Konfigurasi Umum</i>	
Sama dengan Konfigurasi DQN 3.3	

3.4 Pelatihan

Pelatihan dilakukan selama 80,000 episode. Untuk menjamin reproduktibilitas pengukuran performa, *seed global environment* diinisialisasi ke 2004, namun penempatan ranjau (*mine placement*) diacak ulang pada setiap awal episode. Hal ini memastikan *agent* terpapar pada berbagai konfigurasi papan selama pelatihan. Ringkasan konfigurasi pelatihan serta skema pencatatan data disajikan pada Tabel 3.5.

Tabel 3.5. Konfigurasi Pelatihan dan Pencatatan Data

Aspek	Keterangan
Total Episode	80,000
<i>Seed Environment</i>	2004
<i>Seed Model</i>	123
<i>File Logging Per Episode</i>	csv/training_[algoritma].csv
Metrik	<i>episode, total_reward, progress, win, n_clicks, ϵ/β, learn_rate</i>
<i>File Evaluasi Berkala</i>	csv/training_eval_[algoritma].csv
Frekuensi Evaluasi	Setiap 1,000 episode
Metrik Evaluasi	<i>median progress, win rate, median reward, waktu</i>
Penyimpanan Model Terbaik	[algoritma]_best.keras (berdasarkan <i>win rate</i> tertinggi)
<i>Smoothing Kurva</i>	SMA, <i>window</i> 500 episode

3.5 Evaluasi

Evaluasi akhir dilakukan terhadap model terbaik yang disimpan selama pelatihan, yaitu model dengan *win rate* tertinggi pada evaluasi berkala. Pengujian menggunakan 5 *seed* papan yang berbeda (1, 2, 3, 4, 5) untuk mengukur kemampuan generalisasi *agent* pada konfigurasi yang belum pernah dilihat sebelumnya. Ringkasan konfigurasi evaluasi disajikan pada Tabel 3.6.

Selama evaluasi, *agent* tidak melakukan eksplorasi maupun pembelajaran. Seluruh parameter jaringan dibekukan, dan *action* dipilih berdasarkan nilai $Q(s, a)$ tertinggi setelah *masking* petak yang telah terbuka. Hasil evaluasi ini menjadi dasar perhitungan metrik performa akhir seperti *win rate*, rata-rata *total reward*, dan rata-rata jumlah klik.

Tabel 3.6. Konfigurasi Evaluasi Akhir

Aspek	Keterangan
Model yang Digunakan	<code>[algoritma]_best.keras</code> (hasil pelatihan)
<i>Seed</i> Papan Evaluasi	1, 2, 3, 4, 5
Episode per <i>Seed</i>	500
Total Episode Evaluasi	2,500 per <i>agent</i>
<i>Policy</i> Pemilihan <i>Action</i>	<i>Greedy</i> murni ($\epsilon = 0$, tanpa bonus CAE)
<i>File</i> Hasil Evaluasi	<code>csv/eval_[algoritma]_4mines.csv</code>
Metrik	<code>algorithm</code> , <code>board_size</code> , <code>n_mines</code> , <code>eval_seed</code> , <code>episode</code> , <code>win</code> , <code>progress</code> , <code>total_reward</code> , <code>n_clicks</code>

3.6 Metode Analisis

Data yang diperoleh dari pelatihan dan evaluasi dianalisis menggunakan serangkaian uji statistik dan metrik kuantitatif. Rangkuman metode analisis beserta tujuannya disajikan pada Tabel 3.7.

Seluruh pengujian statistik dilakukan pada tingkat signifikansi $\alpha = 0.05$. Uji normalitas *Shapiro-Wilk* diterapkan terlebih dahulu untuk memastikan validitas penggunaan uji parametrik. Metode di atas dipilih agar setiap klaim yang diajukan dalam Bab 4.2, baik mengenai keunggulan performa, kecepatan belajar, stabilitas, maupun kualitas eksplorasi, didukung oleh bukti kuantitatif yang signifikan dan dapat dipertanggungjawabkan secara inferensial.

Tabel 3.7. Metode Analisis Kuantitatif yang Digunakan

Kategori	Metode / Metrik	Tujuan
Performa Akhir	<i>Welch's t-test</i>	Menguji signifikansi perbedaan rata-rata <i>win rate</i> antara DQN dan CAE-DQN
	<i>Cohen's d</i>	Mengukur besaran efek (<i>effect size</i>) perbedaan performa
Interval Kepercayaan	Interval Parametrik	Mengestimasi rentang selisih <i>win rate</i> dengan asumsi normalitas
	<i>Bootstrap</i> (10,000 sampel)	Validasi interval kepercayaan tanpa asumsi distribusi
Efisiensi Sampel	AUC (40,000 episode pertama)	Mengukur total akumulasi performa pada fase awal pelatihan
	Episode mencapai 50% <i>win rate</i>	Indikator kecepatan pembelajaran relatif
Tren Pelatihan	<i>Mann-Kendall & Sen's slope</i>	Mendeteksi keberadaan dan arah tren monoton pada kurva <i>win rate</i>
Stabilitas Akhir	Standar deviasi & regresi linier	Mengukur fluktuasi performa dan signifikansi penurunan pada fase lanjut
Kualitas Eksplorasi	<i>Progress per click</i>	Proporsi klik yang menghasilkan kemajuan (rasio <i>progress</i> terhadap <i>n_clicks</i>)
Adaptivitas Eksplorasi	Analisis bonus CAE	Memvalidasi sifat <i>self-annealing</i> bonus tanpa penjadwalan manual

BAB 4

EKSPERIMEN DAN ANALISIS

Bab ini menyajikan seluruh hasil eksperimen yang telah dilakukan beserta analisis kuantitatif dan kualitatif terhadap data yang diperoleh. Pembahasan diawali dengan pemaparan data mentah dalam bentuk kurva pelatihan, mencakup evolusi *total reward*, jumlah klik, dan *win rate* sepanjang 80,000 episode. Selanjutnya, disajikan kurva evaluasi berkala yang menggambarkan kecepatan pembelajaran relatif kedua *agent* serta visualisasi peluruhan bonus eksplorasi CAE yang menjadi bukti empiris mekanisme *self-annealing*. Data kuantitatif ini kemudian dilengkapi dengan tabel ringkasan metrik evaluasi akhir pada lima *seed* papan yang berbeda, menjadi dasar untuk mengukur performa dan konsistensi *agent*. Setelah pemaparan data, bagian analisis menguraikan hasil uji statistik yang meliputi perbandingan performa akhir, efisiensi sampel, adaptivitas eksplorasi, kualitas eksplorasi, serta stabilitas pelatihan. Setiap sub-analisis didukung oleh uji statistik yang relevan, seperti *Welch's t-test*, *Mann-Kendall*, regresi linier, dan perhitungan ukuran efek, untuk memastikan bahwa kesimpulan yang ditarik tidak hanya bersifat deskriptif tetapi juga memiliki landasan inferensial yang kokoh.

4.1 Data dan Hasil

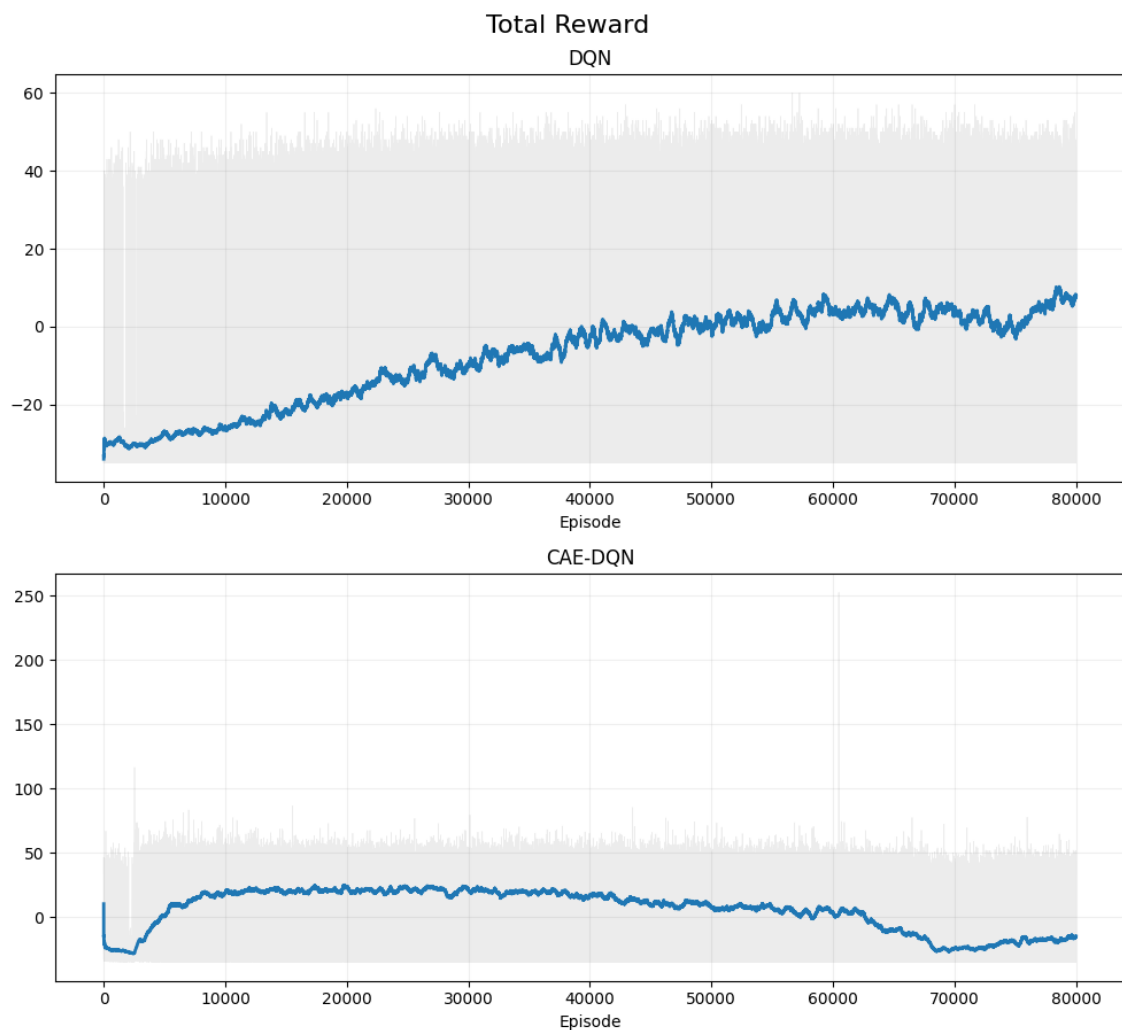
Bagian ini menyajikan hasil dari seluruh eksperimen yang telah dilakukan. Evaluasi difokuskan pada tiga metrik utama: *total reward* per episode, jumlah klik per episode, dan *win rate*. Analisis performa dilakukan dengan membandingkan kurva pelatihan dari DQN standar dan CAE-DQN selama proses *training*, serta ditunjang oleh rangkuman metrik evaluasi akhir untuk mengukur efisiensi sampel dan kestabilan *agent*.

4.1.1 Kurva Pelatihan

Kurva pelatihan menyajikan perkembangan performa *agent* DQN dan CAE-DQN selama 80,000 episode eksperimen. Visualisasi ini mencakup metrik utama: *total reward*, *win rate*, dan *number of clicks* per episode. Seluruh data pada kurva ini diproses menggunakan

metode SMA untuk mengurangi variansi statistik yang tinggi akibat penempatan ranjau secara acak, sehingga tren pembelajaran dan stabilitas konvergensi dari kedua model dapat dianalisis dengan lebih jelas.

Grafik 4.1 menunjukkan perbandingan metrik *total reward* antara model DQN standar dan CAE-DQN selama 80,000 episode, di mana CAE-DQN menunjukkan efisiensi sampel yang jauh lebih tinggi pada fase awal dengan mencapai puncak *reward* sekitar 25 sebelum episode 10,000, jauh melampaui DQN reguler yang baru mencapai nilai positif menjelang akhir pelatihan. Namun, meskipun mampu mencapai performa tinggi dengan sangat cepat, CAE-DQN mengalami fenomena *policy collapse* atau penurunan performa drastis setelah 60,000 episode, di mana nilai *reward* merosot kembali ke level dasar (≈ -30), namun terdapat sedikit tanda-tanda pemulihan setelah episode 70,000.



Gambar 4.1. *Total reward* dari pelatihan DQN dan CAE-DQN selama 80,000 episode

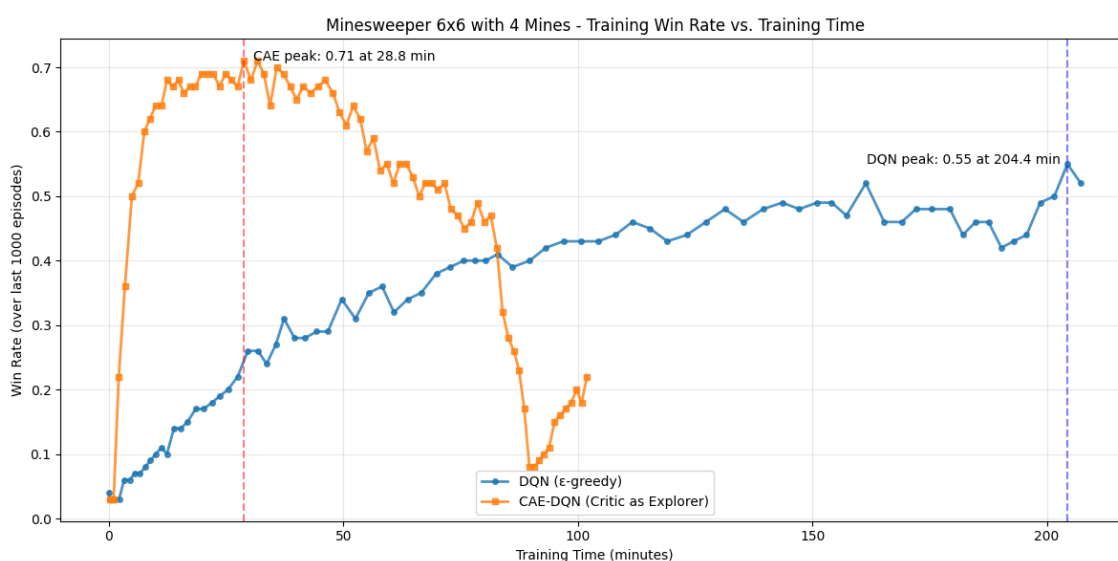
Grafik 4.2 menunjukkan hasil eksperimen jumlah klik selama 80,000 episode dengan trajektori perilaku yang berbeda bagi kedua *agent*. Model DQN standar menunjukkan tren kenaikan yang konsisten dan bertahap, dimulai dari sekitar 4 klik dan terus meningkat hingga mencapai rata-rata puncak hampir 7 klik pada akhir durasi pelatihan. Sebaliknya, CAE-DQN menunjukkan peningkatan jumlah klik yang pesat dalam 10,000 episode pertama, diikuti oleh fase stabil yang berkepanjangan di mana nilai rata-rata berfluktuasi di sekitar 7 klik. Namun, mulai sekitar episode 55,000, CAE-DQN menunjukkan penurunan rata-rata jumlah klik yang cukup terlihat, hingga akhirnya menetap di kisaran 5 klik pada episode 80,000, serta menampilkan variansi puncak yang lebih tinggi pada performa episode individual dibandingkan dengan DQN standar.



Gambar 4.2. *Number of clicks* dari pelatihan DQN dan CAE-DQN selama 80,000 episode

4.1.2 Kurva Evaluasi Pelatihan

Grafik 4.3 menunjukkan perbandingan *win rate* terhadap waktu pelatihan dari model DQN dan CAE-DQN. Grafik menunjukkan perbedaan performa yang signifikan antara DQN dan CAE-DQN pada permainan *Minesweeper* 6×6 . *Agent* CAE-DQN menunjukkan efisiensi pembelajaran yang sangat tinggi dengan lonjakan *win rate* yang tajam di awal pelatihan, mencapai nilai puncak sebesar 71% hanya dalam waktu 28.8 menit. Namun, setelah mencapai fase stabil (*plateau*), performa CAE-DQN mengalami penurunan drastis setelah sekitar menit ke-80, dengan *win rate* terendah dibawah 10%, namun *win rate* menunjukkan pemulihan saat mendekati menit ke-100 (menjelang akhir dari 80,000 episode), mencapai sekitar 20%. Di sisi lain, *agent* DQN menunjukkan proses pembelajaran yang jauh lebih lambat namun stabil secara konsisten. DQN memerlukan waktu sekitar 204.4 menit untuk mencapai performa puncaknya dengan *win rate* sebesar 55%. Meskipun nilai puncak DQN lebih rendah dibandingkan CAE-DQN, DQN tidak menunjukkan degradasi performa yang ekstrem hingga akhir durasi pelatihan yang diamati.

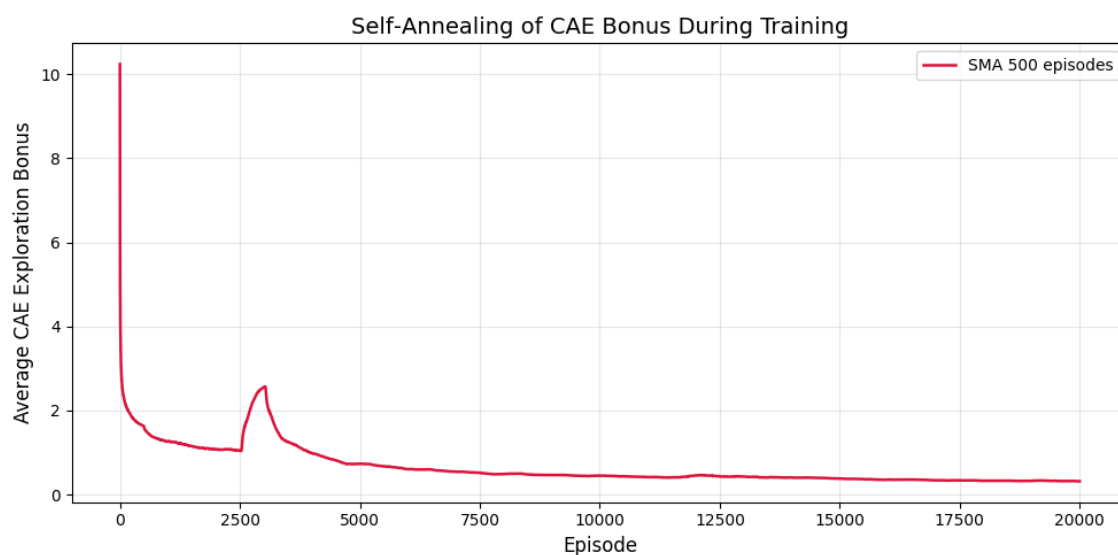


Gambar 4.3. Kurva *win rate* - time dari pelatihan DQN dan CAE-DQN selama 80,000 episode

4.1.3 Bonus Eksplorasi CAE

Grafik 4.4 menampilkan rata-rata bonus eksplorasi CAE per episode yang telah dihaluskan dengan SMA *window* 500 episode. Bonus dimulai dari nilai sekitar 10, menurun

tajam ke kisaran 2 dalam 1,000 episode pertama, kemudian melanjutkan penurunan gradual hingga mencapai sekitar 0,5 pada episode ke-20,000. Sebuah lonjakan kecil teramati di sekitar episode ke-3,000 sebelum bonus kembali menurun secara konsisten.



Gambar 4.4. *Self-annealing* bonus eksplorasi CAE selama 20,000 episode pelatihan.

4.1.4 Metrik Evaluasi

Tabel 4.1 menyajikan rangkuman metrik evaluasi untuk model DQN dan CAE-DQN yang diuji dalam 500 episode untuk setiap *environment seed*. Secara keseluruhan, CAE-DQN menunjukkan performa yang lebih unggul dibandingkan dengan DQN standar pada ketiga metrik utama. CAE-DQN mencapai rata-rata *win rate* sebesar 69.6%, yang secara signifikan lebih tinggi dibandingkan DQN yang hanya mencapai 58.0%. Peningkatan ini juga tercermin pada *average total reward*, di mana CAE-DQN memperoleh nilai 20.073, hampir dua kali lipat dari nilai DQN yang hanya mencapai 11.370. Selain itu, CAE-DQN mencatat rata-rata jumlah klik yang sedikit lebih tinggi, yakni 6.961 klik dibandingkan dengan 6.610 klik pada DQN. Data per *seed* juga menunjukkan konsistensi performa pada kedua model, dengan variasi nilai yang relatif kecil di antara kelima *seed* yang diuji.

Tabel 4.1. Rangkuman metrik evaluasi model DQN dan CAE-DQN sebanyak 500 episode per *seed*

DQN			
<i>Seed</i>	<i>Average Win Rate</i>	<i>Average Total Reward</i>	<i>Average Number of Clicks</i>
1	59.0%	12.318	6.838
2	59.0%	12.058	6.578
3	56.6%	10.272	6.520
4	58.2%	11.530	6.626
5	57.2%	10.670	6.486
Overall	58.0%	11.370	6.610
CAE-DQN			
1	70.8%	21.140	7.164
2	70.8%	20.708	6.732
3	68.8%	19.580	7.044
4	65.6%	17.176	6.944
5	72.0%	21.760	6.920
Overall	69.6%	20.073	6.961

4.2 Analisis

Bagian ini menyajikan analisis mendalam terhadap data yang telah dipaparkan pada bagian sebelumnya, dengan tujuan untuk menjawab pertanyaan penelitian yang diajukan serta mengungkap makna di balik pola-pola kuantitatif yang teramati. Analisis disusun secara bertahap, dimulai dari evaluasi performa akhir untuk mengukur signifikansi statistik keunggulan CAE-DQN atas DQN standar, dilanjutkan dengan pengujian efisiensi sampel melalui metrik AUC dan ambang batas *win rate* 50%. Selanjutnya, perilaku *self-annealing* bonus eksplorasi CAE dianalisis untuk menilai apakah mekanisme intrinsik tersebut mampu menggantikan penjadwalan eksplorasi manual. Kualitas eksplorasi diukur secara kuantitatif melalui rasio *progress per click* pada fase awal pelatihan untuk menentukan apakah *action* yang dihasilkan CAE lebih informatif dibandingkan pendekatan acak ϵ -greedy. Analisis kemudian ditutup dengan pemeriksaan terhadap stabilitas pelatihan jangka panjang, termasuk identifikasi penyebab penurunan performa CAE-DQN pada episode lanjut yang diduga kuat sebagai fenomena *policy collapse*.

Setiap sub-analisis dilengkapi dengan uji statistik yang sesuai, seperti *Welch's t-test*, *Mann-Kendall*, regresi linier, dan perhitungan ukuran efek, sehingga kesimpulan yang dihasilkan tidak hanya bersifat deskriptif tetapi juga memiliki validitas inferensial yang dapat dipertanggungjawabkan.

4.2.1 Perbandingan Performa Akhir

Untuk menguji signifikansi perbedaan performa akhir antara DQN dan CAE-DQN, dilakukan uji statistik terhadap rata-rata *win rate* dari lima *seed* evaluasi yang masing-masing diuji sebanyak 500 episode.

Pertama, dilakukan uji normalitas *Shapiro-Wilk* pada data kedua kelompok. Uji ini bertujuan untuk menentukan apakah distribusi *win rate* per *seed* mengikuti distribusi normal, yang menjadi salah satu asumsi dasar bagi uji-t parametrik. Hasilnya menunjukkan bahwa distribusi *win rate* DQN ($W = 0.888$, $p = 0.346$) dan CAE-DQN ($W = 0.887$, $p = 0.344$) tidak menyimpang secara signifikan dari distribusi normal ($p > 0.05$). Dengan demikian, uji-t independen (*independent samples t-test*) dengan koreksi *Welch* dapat digunakan secara valid untuk membandingkan kedua rata-rata.

Uji-t independen digunakan untuk menguji hipotesis nol bahwa tidak terdapat perbedaan rata-rata *win rate* yang signifikan antara kedua kelompok. Hasil uji-t menunjukkan bahwa CAE-DQN ($M = 69.6\%$, $SD = 2.51\%$) secara signifikan mengungguli DQN ($M = 58.0\%$, $SD = 1.08\%$), $t(5.42) = 9.48$, $p < 0.001$. Dengan demikian, hipotesis nol (H_0) ditolak dan hipotesis alternatif (H_1) diterima, CAE-DQN secara signifikan meningkatkan *win rate* dibandingkan DQN standar.

Untuk mengukur besaran perbedaan secara praktis, dihitung ukuran efek *Cohen's d*, yang mengkuantifikasi seberapa besar perbedaan rata-rata kedua kelompok relatif terhadap variabilitas gabungan data. Nilai *Cohen's d* yang diperoleh sebesar 6.00, yang menurut kriteria konvensional ($d = 0.2$ kecil, $d = 0.5$ sedang, $d = 0.8$ besar) termasuk dalam kategori **sangat besar**. Hal ini mengindikasikan bahwa keunggulan CAE-DQN tidak hanya signifikan secara statistik, tetapi juga sangat substansial secara praktis. **(Catatan:** Karena evaluasi dilakukan pada lima *seed* papan yang berbeda, variabilitas antar *seed* relatif kecil sehingga nilai *Cohen's d* yang diperoleh cenderung tinggi. Meskipun demikian, selisih absolut *win rate* sebesar 11.6% tetap menunjukkan peningkatan per-

forma yang nyata.)

Selanjutnya, untuk mengestimasi rentang nilai selisih *win rate* yang mungkin terjadi pada populasi, dihitung interval kepercayaan 95% parametrik. Interval kepercayaan 95% untuk selisih rata-rata *win rate* adalah [8.53%, 14.67%], yang berarti kita dapat yakin 95% bahwa selisih rata-rata sebenarnya berada dalam rentang tersebut. Sebagai validasi tambahan, dilakukan analisis *bootstrap* dengan 10,000 kali pengambilan sampel ulang, yang menghasilkan interval kepercayaan serupa [9.32%, 13.56%]. Metode *bootstrap* tidak bergantung pada asumsi distribusi normal dan memberikan estimasi interval yang lebih *robust*.

Dengan demikian, berdasarkan seluruh rangkaian uji statistik, dapat disimpulkan bahwa penambahan mekanisme eksplorasi CAE ke dalam arsitektur DQN menghasilkan peningkatan performa akhir yang sangat signifikan dan praktis bermakna (*practically significant*) pada *environment Minesweeper* 6×6 dengan 4 ranjau.

4.2.2 Efisiensi Sampel dan Kecepatan Pembelajaran

Untuk mengukur keunggulan CAE-DQN dalam hal efisiensi sampel (*sample efficiency*) dan kecepatan pembelajaran, dilakukan tiga analisis kuantitatif terhadap data *win rate* yang dicatat setiap 1,000 episode selama pelatihan. Analisis ini mencakup perhitungan AUC, penentuan episode pertama kali mencapai ambang batas 50% *win rate*, serta uji tren *Mann-Kendall*.

Pertama, AUC dihitung dengan mengintegrasikan kurva *win rate* terhadap episode menggunakan metode trapesium pada 40,000 episode pertama. Dalam konteks eksperimen ini, AUC merepresentasikan total akumulasi performa yang berhasil dikumpulkan oleh *agent* selama fase awal pelatihan. Semakin tinggi nilai AUC, semakin cepat *agent* belajar dan semakin tinggi *win rate* yang mampu dipertahankan pada periode tersebut. Hasil perhitungan menunjukkan bahwa CAE-DQN mencapai AUC sebesar 23,400, sementara DQN standar hanya memperoleh AUC sebesar 7,835. Dengan kata lain, CAE-DQN mengakumulasi performa pembelajaran sekitar 2.99 kali lipat lebih besar dibandingkan DQN pada rentang episode yang sama. Perbedaan yang sangat mencolok ini menegaskan bahwa mekanisme eksplorasi berbasis UCB pada CAE secara drastis mempercepat perolehan *reward* dan kemenangan pada fase awal pelatihan.

Kedua, untuk mengukur kecepatan pembelajaran secara lebih intuitif, ditentukan episode pertama di mana masing-masing *agent* berhasil mencapai $\text{win rate} \geq 0.5$ pada evaluasi berkala. CAE-DQN mencapai ambang batas ini pada episode ke-6,000, yang berarti hanya dalam 6,000 episode pertama ($\approx 7.5\%$ dari total pelatihan) *agent* telah mampu memenangkan lebih dari separuh permainan evaluasi. Sebaliknya, DQN baru berhasil menembus ambang batas 50% pada episode ke-65,000, atau setelah menghabiskan lebih dari 80% total episode pelatihan. Perbedaan kecepatan sebesar satu orde magnitudo ini (6,000 vs. 65,000 episode) merupakan bukti kuantitatif yang sangat kuat bahwa CAE memberikan peningkatan efisiensi sampel yang substansial pada *environment Minesweeper* dengan *reward* jarang dan penalti fatal.

Ketiga, untuk mengonfirmasi keberadaan tren peningkatan atau penurunan performa secara statistik sepanjang keseluruhan pelatihan (80,000 episode), dilakukan uji tren non-parametrik *Mann-Kendall*. Uji ini menguji hipotesis nol bahwa tidak terdapat tren monoton (naik atau turun) pada deret data, tanpa mensyaratkan asumsi distribusi tertentu, sehingga *robust* terhadap fluktuasi acak yang umum terjadi pada kurva pembelajaran RL. Hasil uji menunjukkan bahwa DQN memiliki tren yang **meningkat** secara signifikan ($p < 0.001$, *Sen's slope* = 0.0065). Hal ini mengindikasikan bahwa meskipun lambat, DQN secara konsisten menunjukkan perbaikan performa seiring bertambahnya pengalaman. Sebaliknya, CAE-DQN menunjukkan tren yang **menurun** secara signifikan ($p < 0.001$, *Sen's slope* = -0.0059). Tren negatif ini mengonfirmasi secara statistik bahwa performa pelatihan CAE-DQN mengalami degradasi setelah mencapai puncaknya.

Secara keseluruhan, ketiga metrik di atas secara konsisten menunjukkan bahwa CAE-DQN unggul secara dramatis dalam hal efisiensi sampel dan kecepatan pembelajaran awal. Adapun penurunan performa yang teramati pada fase akhir pelatihan CAE-DQN, yang tercermin dari tren *Mann-Kendall* negatif, tidak serta-merta mengindikasikan ketidakstabilan inheren metode CAE. Penurunan tersebut lebih tepat diatribusikan pada fenomena *policy collapse*. Meskipun performa pada papan evaluasi yang berbeda tetap tinggi (Tabel 4.1). Sementara itu, DQN yang belajar lebih lambat belum menunjukkan gejala *policy collapse* hingga akhir pelatihan, mencapai *plateau* yang stabil di sekitar *win rate* 55%.

4.2.3 Adaptivitas Eksplorasi: Self-Annealing Bonus CAE

Penurunan bonus yang teramati pada Grafik 4.4 dapat dijelaskan melalui mekanisme internal CAE. Bonus dihitung sebagai $\beta\sqrt{\phi^\top A^{-1}\phi}$, di mana ϕ adalah vektor fitur laten dari *state* saat ini, dan A adalah *Gram matrix* yang mengakumulasi informasi seluruh *state* yang pernah dikunjungi. Pada awal pelatihan, matriks A hanya berisi regularisasi λI , sehingga invers A^{-1} masih besar dan menghasilkan bonus yang tinggi. Seiring *agent* berinteraksi dengan *environment*, fitur ϕ dari *state* yang sering muncul terus ditambahkan ke dalam A melalui pembaruan *Sherman–Morrison*. Akumulasi ini menyebabkan A^{-1} mengecil secara gradual, yang pada gilirannya menekan nilai bonus.

Penurunan tajam dalam 1,000 episode pertama terjadi karena *agent* dengan cepat mengeksplorasi *state-state* dasar yang mendominasi fase awal permainan. Setelah *state-state* tersebut terwakili dengan baik dalam *Gram matrix*, bonus menurun ke level yang lebih rendah. Lonjakan kecil di sekitar episode ke-3,000 mengindikasikan bahwa *agent* mulai menghadapi konfigurasi papan yang lebih kompleks dan belum pernah ditemui sebelumnya. Karena fitur ϕ dari *state* baru tersebut belum terakumulasi dalam A , ketidakpastian meningkat sementara dan bonus ikut naik. Setelah *state* baru tersebut cukup sering dikunjungi, informasi kembali terakumulasi dan bonus melanjutkan tren penurunan.

Pada akhir 20,000 episode, bonus telah menyusut ke sekitar 0.5, menunjukkan bahwa *agent* telah membangun representasi yang cukup komprehensif terhadap *state space* yang relevan. Dengan demikian, mekanisme CAE secara intrinsik mampu menyesuaikan intensitas eksplorasi berdasarkan tingkat kebaruan *state* yang dihadapi, tanpa memerlukan penjadwalan eksternal.

4.2.4 Kualitas Eksplorasi: Progress per Click

Sebagai indikator kualitas eksplorasi, dihitung metrik *progress per click*, yaitu **proporsi klik yang menghasilkan kemajuan** (rasio antara jumlah klik yang menerima *reward* *progress* terhadap total klik), pada 10,000 episode pertama pelatihan. Metrik ini mengukur seberapa efisien setiap keputusan eksplorasi dalam menambah pengetahuan *agent* tentang papan. Hasil perhitungan menunjukkan bahwa CAE-DQN mencatat rata-rata

progress per click sebesar 0.823, sementara DQN standar memperoleh 0.734. Secara relatif, CAE-DQN mengungkap sekitar 12.1% lebih banyak petak baru per klik dibandingkan DQN pada fase awal pelatihan.

Meskipun peningkatan sebesar 12.1% tampak moderat, perlu dipertimbangkan bahwa pada tahap awal permainan *Minesweeper*, sebagian besar klik secara alami akan membuka sel baru karena sebagian besar papan masih tertutup. Oleh karena itu, ruang untuk perbaikan absolut pada metrik ini memang terbatas. Namun, konsistensi CAE-DQN dalam menghasilkan *progress per click* yang lebih tinggi, terlihat dari median sebesar 0.857 dibandingkan 0.750 pada DQN, mengindikasikan bahwa eksplorasi CAE lebih terarah. *Agent* CAE-DQN cenderung mengklik petak dengan ketidakpastian tinggi yang lebih mungkin membuka area baru di papan.

Dengan demikian, data *progress per click* mendukung pandangan bahwa mekanisme bonus *UCB* pada CAE tidak hanya mempercepat konvergensi secara keseluruhan, tetapi juga meningkatkan kualitas setiap langkah eksplorasi yang diambil, menjadikannya lebih informatif dibandingkan pendekatan ϵ -greedy.

4.2.5 Stabilitas Pelatihan dan Fenomena Penurunan

Untuk menganalisis secara kuantitatif fenomena penurunan performa CAE-DQN yang teramati pada kurva pelatihan setelah episode ke-50,000, dilakukan dua analisis terhadap data *win rate* pada rentang episode 50,000 hingga 80,000: perhitungan variansi sebagai indikator stabilitas, serta regresi linier untuk menguji signifikansi tren penurunan. Perlu dicatat bahwa analisis ini bertujuan untuk mengkarakterisasi dinamika pelatihan dan mendiagnosis penyebab penurunan, bukan untuk mengevaluasi model akhir yang digunakan. Model CAE-DQN yang dievaluasi pada Tabel 4.1 disimpan pada titik performa puncak (sekitar episode ke-6,000–10,000), sehingga pelatihan lanjutan yang dianalisis di sini tidak memengaruhi kualitas *agent* yang diuji pada *seed* berbeda.

Pertama, variansi *win rate* pada 30,000 episode terakhir dihitung untuk menilai kestabilan masing-masing *agent* setelah fase konvergensi awal. Dalam konteks ini, standar deviasi yang lebih rendah mengindikasikan performa yang lebih stabil dan konsisten dari episode ke episode. Hasil perhitungan menunjukkan bahwa DQN memiliki standar deviasi sebesar 0.0297, sedangkan CAE-DQN memiliki standar deviasi sebesar 0.1668, sek-

itar lima kali lipat lebih besar. Perbedaan ini menegaskan bahwa DQN mempertahankan performa yang jauh lebih stabil pada fase akhir pelatihan, sementara CAE-DQN mengalami fluktuasi yang ekstrem seiring dengan penurunan performanya. Rata-rata *win rate* pada rentang yang sama juga mencerminkan hal ini: DQN mempertahankan rata-rata 46.9%, sedangkan CAE-DQN merosot hingga rata-rata 31.6%.

Kedua, untuk menguji apakah penurunan tersebut signifikan secara statistik, dilakukan analisis regresi linier sederhana dengan model $y = \beta_0 + \beta_1 \cdot \text{episode}$ pada data *win rate* CAE-DQN untuk rentang episode 50,000 hingga 80,000. Uji ini bertujuan untuk menentukan apakah terdapat tren linier yang signifikan (koefisien $\beta_1 \neq 0$) pada segmen penurunan. Hasil regresi menunjukkan bahwa koefisien kemiringan (*slope*) $\beta_1 = -0.000016$ bernilai negatif dan signifikan secara statistik ($p < 0.001$), dengan nilai $R^2 = 0.7785$. Temuan ini mengonfirmasi bahwa penurunan *win rate* CAE-DQN pada fase akhir pelatihan bukanlah fluktuasi acak, melainkan tren degradasi yang sistematis dan terprediksi.

Meskipun hasil di atas menunjukkan penurunan performa yang nyata selama pelatihan, penting untuk membedakan antara degradasi pada *environment* pelatihan dan kegagalan metode CAE secara fundamental. Jika penurunan ini disebabkan oleh ketidakstabilan inheren algoritma CAE, maka performa pada *environment* evaluasi yang berbeda seharusnya juga ikut runtuh. Namun, berdasarkan Tabel 4.1, *agent* CAE-DQN yang dievaluasi pada lima *seed* papan berbeda tetap mampu mencapai rata-rata *win rate* sebesar 69.6%. Kesenjangan antara performa pelatihan yang merosot dan performa evaluasi yang tetap tinggi ini mengindikasikan secara kuat bahwa fenomena tersebut merupakan bentuk *policy collapse*, bukan ketidakstabilan sistemis pada metode CAE.

Penjelasan fenomena *collapse* tersebut dapat dikaitkan dengan dinamika *replay buffer*. Dengan kapasitas *buffer* sebesar 300,000 transisi dan rata-rata panjang lintasan sekitar 6~7 langkah per episode, *replay buffer* mencapai kapasitas jenuh tepat pada kisaran episode 45,000 hingga 50,000. Pada titik jenuh ini, mekanisme *First-In-First-Out* (FIFO) mulai melakukan penghapusan terhadap data terlama dalam *buffer*. Data terlama yang dikeluarkan adalah transisi dari fase awal pelatihan (0~10,000 episode), yaitu periode di mana bonus eksplorasi CAE masih sangat tinggi ($b(s) \approx 10$) dan *agent* menghasilkan trajektori yang sangat beragam dalam ruang representasi laten. Saat data lama yang kaya akan var-

iansi ini tergusur sepenuhnya oleh transisi deterministik dari fase *self-annealing* (di mana bonus mendekati 0), distribusi data dalam *buffer* menjadi homogen dan hanya mencerminkan *policy* lokal yang bersifat *greedy*.

Tanpa akses terhadap keragaman pengalaman masa lalu, CNN rentan mengalami *catastrophic forgetting*. Parameter jaringan mulai melupakan fitur-fitur kritis dari area *state space* yang sudah lama tidak dikunjungi, menyebabkan estimasi *Q-value* menjadi *corrupted* dan mendorong *policy* ke dalam *local minimum* yang sub-optimal. Fenomena ini sekaligus menjelaskan mengapa *baseline* DQN tidak mengalami keruntuhan serupa. Strategi ϵ -*greedy* mempertahankan nilai $\epsilon_{\min} = 0,02$, sehingga terdapat injeksi data eksplorasi acak yang konstan ke dalam *buffer*. Injeksi ini bertindak sebagai *regularizer* alami yang mencegah homogenisasi total dari distribusi data dan melindungi stabilitas aproksimasi fungsi jaringan.

Meskipun terjadi penurunan, *agent* menunjukkan pemulihan parsial menjelang akhir pelatihan. Hal ini disebabkan karena bonus eksplorasi CAE tidak pernah mencapai nol, namun masih terdapat nilai residual sekitar 0.5. Bonus residual tersebut dapat mendorong *agent* untuk memilih *action* yang sedikit berbeda dari *greedy policy* yang dominan. Apabila *action* tersebut membawa *agent* ke konfigurasi papan yang sudah lama tidak dikunjungi, pengalaman baru yang masuk ke dalam *replay buffer* akan sedikit mendiversifikasi kembali data pelatihan. Diversifikasi ini memperlambat proses *catastrophic forgetting* dan memberi kesempatan bagi jaringan untuk memperbaiki estimasi *Q-value*, sehingga *win rate* dapat naik kembali secara parsial. Fenomena ini sekaligus memperkuat interpretasi bahwa penyebab utama keruntuhan adalah homogenisasi *buffer*, bukan kegagalan inheren metode CAE.

4.2.6 Ringkasan Temuan Kuantitatif

Sebagai rangkuman dari seluruh analisis yang telah dipaparkan, Tabel 4.2 menyajikan rangkuman dari setiap uji statistik dan metrik kuantitatif yang digunakan untuk membandingkan DQN dan CAE-DQN. Tabel ini dimaksudkan untuk memberikan gambaran menyeluruh secara cepat mengenai bukti empiris yang mendukung keunggulan CAE dalam berbagai aspek yang diukur.

Tabel 4.2. Ringkasan Temuan Kuantitatif: Perbandingan DQN dan CAE-DQN

Aspek	Uji / Hasil	p	Efek	Kesimpulan
Performa Akhir	Welch's t: 69.6% vs 58.0%	<0.001	d=6.00	Unggul signifikan
Efisiensi Sampel	AUC 40k ep: 23,400 vs 7,835	—	2.99×	3× lebih cepat
Kecepatan Belajar	Episode $\geq$ 50% WR: 6k vs 65k	—	—	10× lebih cepat
Tren Pelatihan	Mann-Kendall: <0.001 DQN naik, CAE turun	slope: 0.0065 / -0.0059		DQN naik; CAE <i>collapse</i>
Segmen Penurunan	Regresi 50k–80k: $\beta_1 =$ -0.000016	<0.001	$R^2 = 0.78$	Penurunan signifikan
Stabilitas Akhir	Std. Dev WR: 0.0297 vs 0.1668	—	—	DQN lebih stabil
Adaptivitas Eksplorasi	Bonus CAE: 10 $\rightarrow$ 0.5 (20k ep)	—	—	Self-annealing
Kualitas Eksplorasi	Progress/Klik: 0.823 vs 0.734	—	1.12×	12% lebih informatif

Sebagaimana terangkum dalam Tabel 4.2, seluruh bukti kuantitatif secara konsisten menunjukkan bahwa CAE-DQN mengungguli DQN standar dalam hal kecepatan belajar, efisiensi sampel, dan kualitas eksplorasi. Satu-satunya catatan adalah penurunan performa pada fase pelatihan lanjut yang terindikasi sebagai *policy collapse*, bukan kelemahan inheren metode CAE.

4.3 Pembahasan

Hasil eksperimen secara konsisten mengonfirmasi bahwa implementasi CAE secara signifikan meningkatkan efisiensi sampel dan performa akhir *agent*. Pada *environment* seperti *Minesweeper*, *agent* dihadapkan pada POMDP dengan *reward* yang sangat jarang dan penalti terminal yang fatal. Dalam kondisi ini, strategi eksplorasi acak seperti ϵ -greedy menjadi sangat tidak efisien karena *agent* cenderung memilih *action* yang langsung mengakhiri permainan sebelum sempat mempelajari pola logis *environment*. Sebaliknya, CAE-DQN memandu eksplorasi secara intensional menuju *state* yang belum dipetakan dengan baik oleh jaringan *critic*. Dengan menghindari tebakan acak yang buta, *agent* mampu mempertahankan kelangsungan hidup dalam episode yang lebih panjang pada fase awal pelatihan, secara langsung berdampak pada pencapaian puncak rata-rata *win rate* sebesar 69.6% dalam waktu yang jauh lebih singkat.

Keunggulan performa tersebut tidak terlepas dari mekanisme adaptivitas eksplorasi *agent* yang beroperasi secara *self-annealing*. Secara matematis, bonus eksplorasi CAE didefinisikan sebagai $\beta\sqrt{\phi^T A^{-1} \phi}$, di mana ϕ merepresentasikan fitur laten dari *state* dan A adalah *Gram matrix*. Seiring bertambahnya interaksi *agent* dengan *environment*, fitur laten dari setiap *state* baru ($\phi\phi^T$) ditambahkan ke dalam *Gram matrix* A . Berdasarkan operasi aljabar linier, seiring dengan “terisinya” matriks A oleh berbagai representasi *state*, nilai dari invers *Gram matrix* (A^{-1}) akan menyusut secara monoton. Penyusutan ini menyebabkan bentuk kuadratik $\phi^T A^{-1} \phi$ mengecil secara alami. Fenomena ini membuktikan bahwa jaringan secara matematis “mengetahui” kapan *state space* telah cukup dieksplorasi, memungkinkan *agent* untuk bertransisi dari fase eksplorasi ke eksploitasi tanpa memerlukan skema *decay schedule* manual.

Kualitas eksplorasi yang lebih tinggi juga tercermin pada metrik *progress per click*. Peningkatan metrik ini sebesar 12.1% pada *agent* CAE-DQN menunjukkan bahwa dorongan intrinsik untuk mencari *state* dengan ketidakpastian epistemik tertinggi mendorong *agent* untuk memprioritaskan petak yang memberikan perolehan informasi maksimum. *Agent* secara aktif memilih area perbatasan (*frontier*) yang mengekspos lebih banyak bagian papan permainan. Eksplorasi yang terarah ini mempercepat pemecahan *Constraint Satisfaction Problem* (CSP) yang mendasari logika permainan *Minesweeper*.

Meskipun menunjukkan keunggulan yang substansial pada fase awal hingga menengah, penelitian ini mendapati adanya batasan berupa penurunan performa pada episode pelatihan lanjut (di atas episode 50,000). Fenomena ini disebabkan oleh *policy collapse* akibat hilangnya sinyal eksplorasi. Seiring dengan menyusutnya bonus CAE menuju nol, *agent* bertindak sepenuhnya *greedy* terhadap estimasi *Q-value* dan terjebak dalam *local minimum* pada *policy*-nya. Tanpa adanya mekanisme eksplorasi untuk keluar dari *local minimum* tersebut, *agent* mengulangi kesalahan fatal yang sama secara konsisten, sehingga performa pada *environment* pelatihan merosot tajam. Namun, menjelang akhir pelatihan teramati pemulihan parsial pencapaian *win rate* sampai sekitar 20%. Pemulihan ini dapat diatribusikan pada bonus residual CAE yang tidak pernah sepenuhnya nol, sehingga masih mampu mendorong diversifikasi data di *replay buffer* dan memperlambat proses *catastrophic forgetting*. Meskipun demikian, model yang disimpan pada titik performa puncak, sebelum *collapse* terjadi, tetap menunjukkan kemampuan generalisasi yang unggul pada *seed* evaluasi yang berbeda, dengan rata-rata *win rate* sebesar 69.6%.

BAB 5

PENUTUP

Berdasarkan hasil eksperimen dan analisis, beberapa kesimpulan utama dapat dirumuskan. Pertama, dari sisi validitas implementasi, CAE-DQN berhasil merealisasikan mekanisme eksplorasi berbasis ketidakpastian sesuai dengan formulasi CAE. Pembaruan *Gram matrix* menggunakan *Sherman–Morrison* serta normalisasi *Welford* berjalan stabil sepanjang pelatihan, menunjukkan bahwa metode ini dapat diintegrasikan secara efektif ke dalam kerangka DQN.

Kedua, dari segi efisiensi sampel dan performa, CAE-DQN secara signifikan mengungguli DQN standar, dengan rata-rata *win rate* sebesar 69.6% dibandingkan 58.0% ($p < 0.001$, $d = 6.00$). Selain itu, ambang *win rate* 50% dicapai dalam 6.000 episode, jauh lebih cepat dibandingkan 65.000 episode pada DQN. Perbedaan ini juga tercermin pada AUC (24,300 vs. 7,835), mengindikasikan keunggulan efisiensi pembelajaran secara keseluruhan.

Ketiga, bonus eksplorasi CAE menunjukkan perilaku *self-annealing* yang konsisten, dengan nilai yang menurun dari sekitar 10 pada awal pelatihan menjadi sekitar 0.5 setelah 20,000 episode tanpa memerlukan penjadwalan *decay* manual. Hal ini menunjukkan adanya mekanisme adaptif yang secara alami menggeser proses pembelajaran dari eksplorasi ke eksploitasi.

Keempat, eksplorasi yang dipandu oleh CAE menghasilkan perilaku yang lebih terarah dibandingkan pendekatan ϵ -greedy, sebagaimana ditunjukkan oleh peningkatan metrik *progress per click* sebesar 12.1% pada fase awal pelatihan. *Agent* cenderung memilih *state* dengan ketidakpastian tinggi yang lebih informatif, sehingga mempercepat proses akuisisi pengetahuan tentang lingkungan.

Kelima, penurunan performa pada fase pelatihan lanjut teridentifikasi sebagai *policy collapse* akibat hilangnya sinyal eksplorasi. Seiring bonus CAE menyusut menuju nol, *agent* bertindak sepenuhnya *greedy* dan terjebak dalam *local minimum* pada *policy*-nya. Model yang disimpan pada titik performa puncak (sekitar episode 6,000-10,000), sebelum *collapse* terjadi, tetap menunjukkan kemampuan generalisasi yang baik pada *seed*

evaluasi yang berbeda.

Secara keseluruhan, hasil penelitian ini menunjukkan bahwa CAE merupakan strategi eksplorasi yang efektif untuk *environment* dengan observasi parsial, *reward* jarang, dan penalti terminal, seperti *Minesweeper*. Temuan ini memberikan dasar empiris bagi pengembangan *agent* RL yang lebih adaptif pada domain berbasis logika dengan konsekuensi ireversibel.

5.1 Saran

Berdasarkan temuan dan keterbatasan yang telah diidentifikasi, beberapa saran untuk penelitian selanjutnya dapat dirumuskan sebagai berikut:

1. Pengujian pada Konfigurasi Papan yang Lebih Kompleks

Eksperimen dalam penelitian ini dibatasi pada papan berukuran 6×6 dengan 4 ranjau. Untuk menguji skalabilitas dan generalitas metode CAE-DQN, penelitian selanjutnya dapat mengevaluasi kinerja pada konfigurasi papan yang lebih besar, seperti 9×9 dengan 10 ranjau atau 16×16 dengan 40 ranjau.

2. Penerapan CAE+ pada Environment yang Lebih Sulit

Untuk papan *Minesweeper* yang lebih besar atau *environment* dengan *reward* yang semakin jarang, pembelajaran *value network* menjadi lebih sulit. Penelitian selanjutnya dapat mengimplementasikan CAE+, yang menambahkan jaringan bantu berbasis *Inverse Dynamics* untuk mempercepat pembelajaran representasi laten.

3. Aplikasi pada Domain Logika Diskret Lainnya

Minesweeper merupakan representasi dari lingkungan berbasis logika dengan observasi parsial dan penalti terminal. Penelitian mendatang dapat menerapkan kerangka CAE-DQN pada permainan sejenis seperti *Battleship*, *Mastermind*, atau teka-teki *nonogram*, untuk menguji generalitas temuan ini pada domain yang lebih luas.

4. Studi Ablasi Normalisasi Welford

Normalisasi bonus menggunakan algoritma *Welford* merupakan komponen penting dalam implementasi CAE. Studi ablasi formal yang membandingkan kinerja CAE dengan dan tanpa normalisasi *Welford* pada *environment Minesweeper* akan memberikan pemahaman kuantitatif mengenai kontribusi teknik ini terhadap stabilitas pelatihan.

DAFTAR REFERENSI

- [1] Y. Li *et al.*, “CAE: Repurposing the critic as an explorer in deep reinforcement learning,” *arXiv preprint arXiv:2503.18980*, 2025, also presented at relevant conference.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, “Planning and acting in partially observable stochastic domains,” *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.
- [4] L. Gardea, G. Koontz, and R. Silva, “Training a minesweeper solver,” *CS 229 Project Report, Stanford University*, 2015.
- [5] J. Sherman and W. J. Morrison, “Adjustment of an inverse matrix corresponding to a change in one element of a given matrix,” *The Annals of Mathematical Statistics*, vol. 21, no. 1, pp. 124–127, 1950.
- [6] B. Welford, “Note on a method for calculating corrected sums of squares and products,” *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*, 2nd ed. MIT press, 2018.
- [8] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, “Playing atari with deep reinforcement learning,” *arXiv preprint arXiv:1312.5602*, 2013.
- [9] W. Wang and C. Lei, “Training a minesweeper agent with deep q-network and supervised learning,” *Applied Sciences*, vol. 15, no. 3, p. 745, 2025.

- [10] R. Kaye, “Minesweeper is NP-complete,” *The Mathematical Intelligencer*, vol. 22, no. 2, pp. 9–15, 2000.
- [11] A. Fowler and A. Young, “Minesweeper: A statistical and computational analysis,” *Honors Thesis, Pass with Distinction*, 2004, advisor: Matthew Hudelson.
- [12] M. T. Spaan, “Partially observable Markov decision processes,” in *Reinforcement Learning: State-of-the-Art*. Springer, 2012, pp. 387–414.
- [13] C. Riquelme, G. Tucker, and J. Snoek, “Neural-linear bandits: Overcoming catastrophic forgetting through likelihood matching,” *arXiv preprint arXiv:1801.08612*, 2018.
- [14] P. Xu, Z. Wen, H. Zhao, and Q. Gu, “Neural contextual bandits with deep representation and shallow exploration,” in *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.
- [15] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2015.

LAMPIRAN

Lampiran A – Environment Design

```

1  import random
2  import numpy as np
3
4  class MinesweeperEnv:
5      def __init__(self, width, height, n_mines, seed=None, rewards=None):
6          self.n_rows = width
7          self.n_cols = height
8          self.n_tiles = self.n_rows * self.n_cols
9          self.n_mines = n_mines
10
11         if rewards is None:
12             self.rewards = {
13                 'win': 36,
14                 'lose': -36,
15                 'progress': 1.0,
16             }
17         else:
18             self.rewards = rewards
19
20         self.grid = np.zeros((self.n_rows, self.n_cols), dtype=object)
21         self.board = np.zeros((self.n_rows, self.n_cols), dtype=object)
22         self.state = []
23         self.state_im = np.zeros((self.n_rows, self.n_cols, 10), dtype=np.float32)
24         self.n_clicks = 0
25         self.n_progress = 0
26         self.n_wins = 0
27
28         if seed is not None:
29             self.set_seed(seed)
30
31         self.reset()
32
33     def set_seed(self, seed):
34         random.seed(seed)
35         np.random.seed(seed)
36
37     def reset(self):
38         self.n_clicks = 0
39         self.n_progress = 0
40         self.n_wins = 0
41         self._init_grid()

```

```

42     self._compute_board()
43     self._init_state()
44     return self.state_im
45
46 def _init_grid(self):
47     self.grid.fill(None)
48     mines_placed = 0
49     while mines_placed < self.n_mines:
50         r = random.randint(0, self.n_rows - 1)
51         c = random.randint(0, self.n_cols - 1)
52         if self.grid[r, c] != 'B':
53             self.grid[r, c] = 'B'
54             mines_placed += 1
55
56 def _get_neighbors(self, row, col):
57     neighbors = []
58     for dr in (-1, 0, 1):
59         for dc in (-1, 0, 1):
60             if dr == 0 and dc == 0:
61                 continue
62             nr, nc = row + dr, col + dc
63             if 0 <= nr < self.n_rows and 0 <= nc < self.n_cols:
64                 neighbors.append(self.grid[nr, nc])
65     return neighbors
66
67 def _count_adjacent_mines(self, row, col):
68     neighbors = self._get_neighbors(row, col)
69     return sum(1 for v in neighbors if v == 'B')
70
71 def _compute_board(self):
72     self.board = self.grid.copy()
73     for r in range(self.n_rows):
74         for c in range(self.n_cols):
75             if self.board[r, c] != 'B':
76                 self.board[r, c] = self._count_adjacent_mines(r, c)
77
78 def _init_state(self):
79     self.state = []
80     for r in range(self.n_rows):
81         for c in range(self.n_cols):
82             self.state.append({'coord': (r, c), 'value': 'U'})
83     self._update_state_image()
84
85 def _one_hot_cell(self, value):
86     one_hot = [0.0] * 10
87     if value == 'U':

```

```

88         one_hot[9] = 1.0
89         elif isinstance(value, (int, float)) and 0 <= value <= 8:
90             one_hot[int(value)] = 1.0
91         else:
92             one_hot[9] = 1.0
93         return one_hot
94
95     def _update_state_image(self):
96         im = np.zeros((self.n_rows, self.n_cols, 10), dtype=np.float32)
97         for idx, tile in enumerate(self.state):
98             r = idx // self.n_cols
99             c = idx % self.n_cols
100             vec = self._one_hot_cell(tile['value'])
101             im[r, c, :] = vec
102         self.state_im = im
103
104     def _reveal_cell(self, row, col):
105         idx = row * self.n_cols + col
106         self.state[idx]['value'] = self.board[row, col]
107
108     def _reveal_neighbors(self, row, col, processed):
109         processed.append((row, col))
110         for dr in (-1, 0, 1):
111             for dc in (-1, 0, 1):
112                 if dr == 0 and dc == 0:
113                     continue
114                 nr, nc = row + dr, col + dc
115                 if (0 <= nr < self.n_rows and 0 <= nc < self.n_cols and
116                     (nr, nc) not in processed):
117                     self._reveal_cell(nr, nc)
118                     if self.board[nr, nc] == 0:
119                         self._reveal_neighbors(nr, nc, processed)
120
121     def click(self, action_index):
122         r = action_index // self.n_cols
123         c = action_index % self.n_cols
124         value = self.board[r, c]
125
126         if value == 'B' and self.n_clicks == 0:
127             safe_indices = []
128             for i in range(self.n_tiles):
129                 rr = i // self.n_cols
130                 cc = i % self.n_cols
131                 if self.board[rr, cc] != 'B':
132                     safe_indices.append(i)
133             new_idx = random.choice(safe_indices)

```

```

134         r = new_idx // self.n_cols
135         c = new_idx % self.n_cols
136         value = self.board[r, c]
137         self._reveal_cell(r, c)
138     else:
139         self._reveal_cell(r, c)
140
141     if value == 0:
142         self._reveal_neighbors(r, c, [])
143
144     self.n_clicks += 1
145
146     def step(self, action_index):
147         prev_unknown = np.sum(self.state_im[:, :, 9] == 1)
148
149         r = action_index // self.n_cols
150         c = action_index % self.n_cols
151         neighbors = self._get_neighbors(r, c)
152
153         self.click(action_index)
154         self._update_state_image()
155
156         done = False
157         reward = 0
158
159         if self.state[action_index]['value'] == 'B':
160             reward = self.rewards['lose']
161             done = True
162
163         elif np.sum(self.state_im[:, :, 9] == 1) == self.n_mines:
164             reward = self.rewards['win']
165             done = True
166             self.n_progress += 1
167             self.n_wins += 1
168
169         elif np.sum(self.state_im[:, :, 9] == 1) == prev_unknown:
170             reward = self.rewards['no_progress']
171
172         else:
173             if all(n == 'U' for n in neighbors):
174                 reward = self.rewards['guess']
175             else:
176                 reward = self.rewards['progress']
177                 self.n_progress += 1
178
179         return self.state_im, reward, done

```



```

180
181 def draw_state(self):
182     grid_display = np.full((self.n_rows, self.n_cols), '?', dtype=str)
183     for idx, tile in enumerate(self.state):
184         r = idx // self.n_cols
185         c = idx % self.n_cols
186         val = tile['value']
187         if val == 'U':
188             grid_display[r, c] = '?'
189         elif val == 'B':
190             grid_display[r, c] = 'B'
191         else:
192             grid_display[r, c] = str(val)
193     print(grid_display)

```

Lampiran B - Desain DQN

dqn_model.py

```

1 # encoder_dqn/dqn/dqn_model.py
2 from keras.models import Sequential
3 from keras.layers import Conv2D, Dense, Flatten, Input
4 from keras.optimizers import Adam
5
6 def create_dqn(learning_rate, input_shape, num_actions, conv_units=64,
7     ↪ dense_units=256):
8     model = Sequential([
9         Input(shape=input_shape),
10        Conv2D(conv_units, (3, 3), activation='relu', padding='same'),
11        Conv2D(conv_units, (3, 3), activation='relu', padding='same'),
12        Conv2D(conv_units, (3, 3), activation='relu', padding='same'),
13        Flatten(),
14        Dense(dense_units, activation='relu'),
15        Dense(dense_units, activation='relu'),
16        Dense(num_actions, activation='linear')
17    ])
18    model.compile(optimizer=Adam(learning_rate=learning_rate, epsilon=1e-4),
19        ↪ loss='mse')
20    return model

```

dqn_agent.py

```

1  import random
2  import numpy as np
3  from collections import deque
4  from dqn_model import create_dqn
5  import tensorflow as tf
6
7  MEMORY_SIZE = 300_000
8  MEMORY_MIN = 10_000
9  BATCH_SIZE = 128
10 LEARNING_RATE = 0.001
11 LEARNING_DECAY = 1.0
12 DISCOUNT_FACTOR = 0.9
13 EPSILON = 0.95
14 EPSILON_DECAY = 0.99995
15 EPSILON_MIN = 0.02
16 UPDATE_TARGET_EVERY = 1000
17 CONV_UNITS = 32
18 DENSE_UNITS = 128
19 TRAIN_EVERY = 4
20
21 MODEL_NAME = f'conv{CONV_UNITS}x3_dense{DENSE_UNITS}x2_y{DISCOUNT_FACTOR}'
22
23 class DQNAgent:
24     def __init__(self, env, model_name=MODEL_NAME, seed=None,
25                 conv_units=CONV_UNITS, dense_units=DENSE_UNITS):
26         self.env = env
27         self.model_name = model_name
28
29         if seed is not None:
30             random.seed(seed)
31             np.random.seed(seed)
32
33         self.discount = DISCOUNT_FACTOR
34         self.learning_rate = LEARNING_RATE
35         self.epsilon = EPSILON
36         self.batch_size = BATCH_SIZE
37
38         self.train_every = TRAIN_EVERY
39         self.train_step_counter = 0
40
41         input_shape = self.env.state_im.shape
42         num_actions = self.env.n_tiles
43
44         self.model = create_dqn(self.learning_rate, input_shape, num_actions,

```

```

45         conv_units, dense_units)
46     self.target_model = create_dqn(self.learning_rate, input_shape,
47     ↪ num_actions,
48         conv_units, dense_units)
49     self.target_model.set_weights(self.model.get_weights())
50
51     self.replay_memory = deque(maxlen=MEMORY_SIZE)
52     self.target_update_counter = 0
53
54     def get_action(self, state):
55         unknown_channel = state[:, :, 9]
56         flat_unknown = unknown_channel.reshape(-1)
57         unsolved = [i for i, u in enumerate(flat_unknown) if u == 1.0]
58
59         if np.random.random() < self.epsilon:
60             action = np.random.choice(unsolved)
61         else:
62             input_batch = np.expand_dims(state, axis=0)
63             q_values = self.model.predict_on_batch(input_batch)[0]
64             mask = (flat_unknown == 0)
65             q_values[mask] = np.min(q_values)
66             action = np.argmax(q_values)
67         return action
68
69     def update_replay_memory(self, transition):
70         self.replay_memory.append(transition)
71
72     @tf.function
73     def _train_step(self, states, actions, rewards, next_states, dones):
74         current_qs = self.model(states, training=False)
75         future_qs = self.target_model(next_states, training=False)
76         max_future_q = tf.reduce_max(future_qs, axis=1)
77
78         targets = rewards + self.discount * max_future_q * (1.0 - tf.cast(dones,
79     ↪ tf.float32))
80
81         with tf.GradientTape() as tape:
82             q_values = self.model(states, training=True)
83             actions_one_hot = tf.one_hot(actions, depth=self.env.n_tiles)
84             selected_q = tf.reduce_sum(q_values * actions_one_hot, axis=1)
85             loss = tf.reduce_mean(tf.square(targets - selected_q))
86
87         grads = tape.gradient(loss, self.model.trainable_variables)
88         self.model.optimizer.apply_gradients(zip(grads,
89     ↪ self.model.trainable_variables))
90         return loss

```

```

89 def train(self, done):
90     self.train_step_counter += 1
91     if self.train_step_counter % self.train_every != 0:
92         return
93
94     if len(self.replay_memory) < MEMORY_MIN:
95         return
96
97     batch = random.sample(self.replay_memory, self.batch_size)
98
99     state_shape = self.env.state_im.shape
100     X = np.empty((self.batch_size, *state_shape), dtype=np.float32)
101     actions_arr = np.empty(self.batch_size, dtype=np.int32)
102     rewards_arr = np.empty(self.batch_size, dtype=np.float32)
103     next_states_arr = np.empty((self.batch_size, *state_shape),
104     ↪ dtype=np.float32)
105     dones_arr = np.empty(self.batch_size, dtype=np.bool_)
106
107     for i, (state, action, reward, next_state, terminal) in enumerate(batch):
108         X[i] = state
109         actions_arr[i] = action
110         rewards_arr[i] = reward
111         next_states_arr[i] = next_state
112         dones_arr[i] = terminal
113
114     self._train_step(
115         tf.convert_to_tensor(X, dtype=tf.float32),
116         tf.convert_to_tensor(actions_arr, dtype=tf.int32),
117         tf.convert_to_tensor(rewards_arr, dtype=tf.float32),
118         tf.convert_to_tensor(next_states_arr, dtype=tf.float32),
119         tf.convert_to_tensor(dones_arr, dtype=tf.bool)
120     )
121
122     self.target_update_counter += 1
123     if self.target_update_counter >= UPDATE_TARGET_EVERY:
124         self.target_model.set_weights(self.model.get_weights())
125         self.target_update_counter = 0
126
127     self.epsilon = max(EPSILON_MIN, self.epsilon * EPSILON_DECAY)

```

Lampiran B - Desain CAE-DQN

caedqn_model.py

```

1 import tensorflow as tf
2 from tensorflow.keras import layers, Model #type: ignore
3
4 def create_caedqn_model(learning_rate, input_shape, num_actions, conv_units=32,
5   ↪ dense_units=128):
6     state_input = layers.Input(shape=input_shape, name='state')
7
8     x = layers.Conv2D(conv_units, (3, 3), activation='relu',
9   ↪ padding='same')(state_input)
10    x = layers.Conv2D(conv_units, (3, 3), activation='relu', padding='same')(x)
11    x = layers.Conv2D(conv_units, (3, 3), activation='relu', padding='same')(x)
12    x = layers.Flatten()(x)
13
14    x = layers.Dense(dense_units, activation='relu')(x)
15
16    embedding = layers.Dense(dense_units, activation='relu', name='embedding')(x)
17
18    q_values = layers.Dense(num_actions, activation='linear',
19   ↪ name='q_values')(embedding)
20
21    model = Model(inputs=state_input, outputs=[q_values, embedding])
22
23    model.compile(
24        optimizer=tf.keras.optimizers.Adam(learning_rate=learning_rate,
25   ↪ epsilon=1e-4),
26        loss={'q_values': 'mse'}
27    )
28    return model

```

caedqn_agent.py

```

1 import random
2 import numpy as np
3 from collections import deque
4 import tensorflow as tf
5 from caedqn_model import create_caedqn_model
6
7 MEMORY_SIZE = 300_000
8 MEMORY_MIN = 10_000
9 BATCH_SIZE = 128

```

```

10 LEARNING_RATE = 0.001
11 DISCOUNT_FACTOR = 0.9
12 UPDATE_TARGET_EVERY = 1000
13 CONV_UNITS = 32
14 DENSE_UNITS = 128
15 TRAIN_EVERY = 4
16
17 BETA = 0.1
18 RIDGE = 1.0
19 RECALC_INV_EVERY = 10000
20
21 MODEL_NAME = f'caedqn_conv{CONV_UNITS}x3_dense{DENSE_UNITS}x2'
22
23 class CAEDQNAgent:
24     def __init__(self, env, model_name=MODEL_NAME, seed=None,
25         ↪ conv_units=CONV_UNITS, dense_units=DENSE_UNITS, beta=BETA, ridge=RIDGE):
26         self.env = env
27         self.model_name = model_name
28         self.beta = beta
29         self.ridge = ridge
30         self.embedding_dim = dense_units
31         self.step_count = 0
32
33         if seed is not None:
34             random.seed(seed)
35             np.random.seed(seed)
36             tf.random.set_seed(seed)
37
38         self.discount = DISCOUNT_FACTOR
39         self.learning_rate = LEARNING_RATE
40         self.batch_size = BATCH_SIZE
41         self.train_every = TRAIN_EVERY
42         self.train_step_counter = 0
43
44         input_shape = self.env.state_im.shape
45         num_actions = self.env.n_tiles
46
47         self.model = create_caedqn_model(self.learning_rate, input_shape,
48             ↪ num_actions, conv_units, dense_units)
49         self.target_model = create_caedqn_model(self.learning_rate, input_shape,
50             ↪ num_actions, conv_units, dense_units)
51         self.target_model.set_weights(self.model.get_weights())
52
53         self.A = np.eye(self.embedding_dim) * self.ridge
54         self.A_inv = np.eye(self.embedding_dim) / self.ridge
55
56         self.bonus_mean = 0.0

```

```

54     self.bonus_var = 0.0
55     self.bonus_count = 0
56
57     self.replay_memory = deque(maxlen=MEMORY_SIZE)
58     self.target_update_counter = 0
59
60     def get_action(self, state):
61         unknown_channel = state[:, :, 9]
62         flat_unknown = unknown_channel.reshape(-1)
63
64         state_batch = np.expand_dims(state, axis=0)
65         q_values, embedding = self.model.predict_on_batch(state_batch)
66         q_values = q_values[0]
67         phi = embedding[0]
68
69         A_inv_phi = np.dot(self.A_inv, phi)
70         raw_bonus = self.beta * np.sqrt(np.dot(phi, A_inv_phi))
71
72         self.bonus_count += 1
73         delta = raw_bonus - self.bonus_mean
74         self.bonus_mean += delta / self.bonus_count
75         delta2 = raw_bonus - self.bonus_mean
76         self.bonus_var += delta * delta2
77         std = np.sqrt(self.bonus_var / self.bonus_count) if self.bonus_count > 1
78         ↪ else 1.0
79         scaled_bonus = raw_bonus / (std + 1e-8)
80
81         augmented_q = q_values + scaled_bonus
82
83         mask = (flat_unknown == 0)
84         augmented_q[mask] = -np.inf
85
86         action = int(np.argmax(augmented_q))
87
88         denom = 1.0 + np.dot(phi, A_inv_phi)
89         numerator = np.outer(A_inv_phi, A_inv_phi)
90         self.A_inv -= numerator / denom
91         self.A += np.outer(phi, phi)
92
93         self.step_count += 1
94         if self.step_count % RECALC_INV_EVERY == 0:
95             self.A_inv = np.linalg.inv(self.A)
96
97         return action
98
99     def compute_bonus(self, state):
100         state_batch = np.expand_dims(state, axis=0)

```

```

100     _, embedding = self.model.predict_on_batch(state_batch)
101     phi = embedding[0]
102
103     A_inv_phi = np.dot(self.A_inv, phi)
104     raw_bonus = self.beta * np.sqrt(np.dot(phi, A_inv_phi))
105
106     if self.bonus_count > 1:
107         std = np.sqrt(self.bonus_var / self.bonus_count)
108     else:
109         std = 1.0
110     scaled_bonus = raw_bonus / (std + 1e-8)
111     return float(scaled_bonus)
112
113 def update_replay_memory(self, transition):
114     self.replay_memory.append(transition)
115
116 @tf.function
117 def _train_step(self, states, actions, rewards, next_states, done):
118     q_values, _ = self.model(states, training=True)
119
120     future_qs, _ = self.target_model(next_states, training=False)
121     max_future_q = tf.reduce_max(future_qs, axis=1)
122     targets = rewards + self.discount * max_future_q * (1.0 - tf.cast(done,
123     ↪ tf.float32))
124
125     with tf.GradientTape() as tape:
126         q_values, _ = self.model(states, training=True)
127         actions_one_hot = tf.one_hot(actions, depth=self.env.n_tiles)
128         selected_q = tf.reduce_sum(q_values * actions_one_hot, axis=1)
129         loss = tf.reduce_mean(tf.square(targets - selected_q))
130
131     grads = tape.gradient(loss, self.model.trainable_variables)
132     self.model.optimizer.apply_gradients(zip(grads,
133     ↪ self.model.trainable_variables))
134     return loss
135
136 def train(self, done):
137     self.train_step_counter += 1
138     if self.train_step_counter % self.train_every != 0:
139         return
140     if len(self.replay_memory) < MEMORY_MIN:
141         return
142
143     batch = random.sample(self.replay_memory, self.batch_size)
144
145     state_shape = self.env.state_im.shape
146     X = np.empty((self.batch_size, *state_shape), dtype=np.float32)

```



```

145     actions_arr = np.empty(self.batch_size, dtype=np.int32)
146     rewards_arr = np.empty(self.batch_size, dtype=np.float32)
147     next_states_arr = np.empty((self.batch_size, *state_shape),
    ↪ dtype=np.float32)
148     dones_arr = np.empty(self.batch_size, dtype=np.bool_)
149
150     for i, (state, action, reward, next_state, terminal) in enumerate(batch):
151         X[i] = state
152         actions_arr[i] = action
153         rewards_arr[i] = reward
154         next_states_arr[i] = next_state
155         dones_arr[i] = terminal
156
157     self._train_step(
158         tf.convert_to_tensor(X, dtype=tf.float32),
159         tf.convert_to_tensor(actions_arr, dtype=tf.int32),
160         tf.convert_to_tensor(rewards_arr, dtype=tf.float32),
161         tf.convert_to_tensor(next_states_arr, dtype=tf.float32),
162         tf.convert_to_tensor(dones_arr, dtype=tf.bool)
163     )
164
165     self.target_update_counter += 1
166     if self.target_update_counter >= UPDATE_TARGET_EVERY:
167         self.target_model.set_weights(self.model.get_weights())
168         self.target_update_counter = 0

```